Synopsys IP Basic Core Module Synchronizers

Application Note

Version 2024.12
December 2024

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History 5

Preface 7

Chapter 1
Synchronizer Modules 11
1.1 Summary of BCM Synchronizers Used in Synthesizable IP 12
1.2 Preserving Registers in BCMs During Synthesis 14
1.3 Synchronizer 1: Simple Multiple Register Synchronizer 15
1.3.1 Synchronizer 1 for Automotive IP 18
1.4 Synchronizer 2: Pulse Synchronizer 19
1.4.1 Pulse Mode / Send Mode Filter for Synchronizer 2 19
1.4.2 Synchronizer 2 for Automotive IP 20
1.5 Synchronizer 3: Pulse Synchronizer with Acknowledge 20
1.5.1 Pulse Mode / Send Mode Filter for Synchronizer 3 21
1.5.2 Synchronizer 3 for Automotive IP 21
1.6 Synchronizer 4: Gray Coded Synchronizer 22
1.6.1 Synchronizer 4 for Automotive IP 22
1.7 Synchronizer 5: Data Bus Synchronizer with Acknowledge 23
1.7.1 Pulse Mode / Send Mode Filter 24
1.7.2 Synchronizer 5 for Automotive IP 24
1.8 Synchronizer 6: Data Bus Synchronizer without Acknowledge 25
1.8.1 Pulse Mode / Send Mode Filter 25
1.8.2 Synchronizer 6 for Automotive IP 26
1.9 Synchronizer 7: Quasi-Synchronous Data Bus Synchronizer from Low Freq to High Freq 26
1.10 Synchronizer 8: Quasi-Synchronous Data Bus Synchronizer from High Frequency to Low Frequency . 28
1.11 Synchronizer 9: Single Clock Data Bus Synchronizer 31
1.11.1 Synchronizer 9 for Automotive IP 31
1.12 Synchronizer 10: Data Stream Synchronizer 32
1.13 Synchronizer 11: Dual Clock FIFO Controller 33
1.13.1 Synchronizer 11 as <IP_NAME>_bcm07, <IP_NAME>_bcm87, or <IP_NAME>_bcm66 33
1.13.2 Synchronizer 11 as <IP_NAME>_bcm38 or <IP_NAME>_bcm74 34
1.13.3 RAMs for Synchronizer 11 36
1.13.4 Synchronizer 11 for Automotive IP 37
1.14 Synchronizer 12: Reset Sequence Synchronizer 38
1.14.1 Synchronizer 12 for Automotive IP 38
1.15 Synchronizer 13: Simple Reset Synchronizer 39
1.16 Synchronizer 14: Reset Synchronizer with Clock Gating 40

1.17 Synchronizer 15: Compound Dual Clock FIFO 42
1.18 TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP 44

Revision History

Version	Date	Description
2024.12	December 2024	■ Added new BCM <IP_NAME>_bcm21_inj to Synchronizer 1 in Table 1-1 on page 12 and to the “Implemented In” column in Table 1-2 on page 14 ■ Added BCM <IP_NAME>_bcm23_c to Synchronizer 3 in Table 1-1 on page 12 and in “Synchronizer 3: Pulse Synchronizer with Acknowledge” on page 20 ■ Added BCM <IP_NAME>_bcm25_c to Synchronizer 5 in Table 1-1 on page 12 and in “Synchronizer 5: Data Bus Synchronizer with Acknowledge” on page 23 ■ Added BCM <IP_NAME>_bcm87 to Synchronizer 11 in Table 1-1 on page 12 and in “Synchronizer 11: Dual Clock FIFO Controller” on page 33 ■ Corrected a typo in the text in Figure 1-4 on page 16 and in Figure 1-5 on page 17 ■ Removed obsolete usage of the *VERIF_EN* parameter from “Synchronizer 9: Single Clock Data Bus Synchronizer” on page 31 ■ Removed obsoleted missampling type 3 from Figure 1-6 on page 17 ■ Removed obsoleted missampling type 2 from Figure 1-7 on page 18
2024.09-SP1	November 2024	■ Updated date and version; no technical updates
2024.09	September 2024	■ Added “Preserving Registers in BCMs During Synthesis” on page 14 ■ Added TMR self-test details to text and illustrations in “TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP” on page 44
2024.06-SP1	July 2024	■ Added new BCM components to Table 1-1 on page 12 ■ Added new BCM components for Automotive IP for: - “Synchronizer 4: Gray Coded Synchronizer” on page 22 - “Synchronizer 11: Dual Clock FIFO Controller” on page 33 - “Synchronizer 12: Reset Sequence Synchronizer” on page 38 ■ For Synchronizer 1, replaced diagrams for missampling models 2 and 3 in Figure 1-6 on page 17 and Figure 1-7 on page 18
2024.06	June 2024	■ Updated Table 1-1 on page 12 and added new BCM components ■ Made general changes in presentation for BCMs that support automotive IP ■ Added “TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP” on page 44
2024.03-SP1	April 2024	Initial publication of this document

Preface

Application Note Organization

This application note describes a set of Synopsys IP Basic Core Module (BCM) designs that are typically used to cross clock boundaries in Synopsys synthesizable IP. The document is organized as follows:

- Chapter 1, "Synchronizer Modules" contains the following structure:
  - Table 1-1 on page 12 lists the synchronizer BCMs.
  - Links in Table 1-1 on page 12 guide you to sections where each BCM is described.

Related Documentation

For help with the Synopsys coreConsultant and coreAssembler tools, refer to the following documentation:

- *coreConsultant User Guide*
- *coreAssembler User Guide*

Web Resources

For more information, check the following Synopsys online resources:

- Synopsys IP product information: https://www.synopsys.com/designware-ip.html
- Your custom Synopsys IP page: https://www.synopsys.com/dw/mydesignware.php
- Documentation through SolvNetPlus: https://solvnetplus.synopsys.com (SolvNetPlus password required)
- Synopsys Common Licensing (SCL): https://www.synopsys.com/keys

STAR on the Web (SotW)

You must review all STARs on the Web (SotWs) associated with your product. SotWs are considered a part of the Synopsys documentation suite, and show critical information related to your product. To review product SotWs, refer to the Synopsys IP product information:

https://www.synopsys.com/designware-ip.html

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

To obtain support for your product, prepare the required files and contact the support center using one of the methods described:

- Prepare the following debug information, if applicable:
  - For environment set-up problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, select the following menu:

    File > Build Debug Tar-file

    Check all the boxes in the dialog box that apply to your issue. This option gathers all the Synopsys product data needed to begin debugging an issue and writes it to the <*core tool startup directory*>/debug.tar.gz file.
  - For simulation issues outside of coreConsultant or coreAssembler:
    - Create a waveforms file (such as VPD or VCD).
    - Identify the hierarchy path to the Synopsys IP instance.
    - Identify the timestamp of any signals or locations in the waveforms that are not understood.
- *For the fastest response*, enter a case through SolvNetPlus:

  a. https://solvnetplus.synopsys.com

  Note SolvNetPlus does not support Internet Explorer.

  b. Click the **Cases** menu and then click **Create a New Case** (below the list of cases).

  c. Complete the mandatory fields that are marked with an asterisk and click **Save**.

     Make sure to include the following:
     - **Product L1:** DesignWare Cores
     - **Product L2:** <name of L2>

  d. After creating the case, attach any debug files you created.

  For more information about general usage information, refer to the following article in SolvNetPlus:

  https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources

- Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
  - Include the Product L1 and Product L2 names, and Version number in your e-mail so it can be routed correctly.
  - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood
  - Attach any debug files you created.
- Or, telephone your local support center:
  - North America:

    Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
  - All other countries:

    https://www.synopsys.com/support/global-support-centers.html

1
Synchronizer Modules

This document describes the Synopsys IP Basic Core Module (BCM) synchronizers that are typically used to cross clock boundaries in Synopsys synthesizable IP. These verified BCM RTL designs are organized into various types of synchronizers.

This chapter contains the following major topics:

- "Summary of BCM Synchronizers Used in Synthesizable IP" on page 12
- "Preserving Registers in BCMs During Synthesis" on page 14
- "Synchronizer 1: Simple Multiple Register Synchronizer" on page 15
- "Synchronizer 2: Pulse Synchronizer" on page 19
- "Synchronizer 3: Pulse Synchronizer with Acknowledge" on page 20
- "Synchronizer 4: Gray Coded Synchronizer" on page 22
- "Synchronizer 5: Data Bus Synchronizer with Acknowledge" on page 23
- "Synchronizer 6: Data Bus Synchronizer without Acknowledge" on page 25
- "Synchronizer 7: Quasi-Synchronous Data Bus Synchronizer from Low Freq to High Freq" on page 26
- "Synchronizer 8: Quasi-Synchronous Data Bus Synchronizer from High Frequency to Low Frequency" on page 28
- "Synchronizer 9: Single Clock Data Bus Synchronizer" on page 31
- "Synchronizer 10: Data Stream Synchronizer" on page 32
- "Synchronizer 11: Dual Clock FIFO Controller" on page 33
- "Synchronizer 12: Reset Sequence Synchronizer" on page 38
- "Synchronizer 13: Simple Reset Synchronizer" on page 39
- "Synchronizer 14: Reset Synchronizer with Clock Gating" on page 40
- "Synchronizer 15: Compound Dual Clock FIFO" on page 42
- "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44

1.1 Summary of BCM Synchronizers Used in Synthesizable IP

In Table 1-1, the BCM RTL files for the synchronizers are listed and linked to where the corresponding synchronizer type is described. Note that some BCM designs contain other BCM designs as sub-modules in a building-block fashion. Also, where you see an asterisk (*) in Table 1-1 (and throughout this document), several uniquely named forms of the BCM are available, and any of them might be used in the Synopsys IP to provide the corresponding synchronizer function. For example, "<IP_NAME>_bcm21*.v" refers to either:

- <IP_NAME>_bcm21.v,
- <IP_NAME>_bcm21_inj.v,
- <IP_NAME>_bcm21_neo.v, or
- <IP_NAME>_bcm21_tgl.v.

Table 1-1 BCM Synchronizers Used in Synthesizable IP

Synchronizer Type and Number	BCM RTL File	Submodules
Synchronizer 1: Simple Multiple Register Synchronizer	<IP_NAME>_bcm21*.v	N/A
	<IP_NAME>_bcm21_atv.v	<IP_NAME>_bcm21.v <IP_NAME>_bcm21_inj.v
Synchronizer 1: Simple Multiple Register Synchronizer (with configurable reset)	<IP_NAME>_bcm41*.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm41_atv.v	<IP_NAME>_bcm21.v
Synchronizer 2: Pulse Synchronizer	<IP_NAME>_bcm22.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm36.v	<IP_NAME>_bcm22.v
	<IP_NAME>_bcm22_atv.v	<IP_NAME>_bcm21_atv.v
Synchronizer 3: Pulse Synchronizer with Acknowledge	<IP_NAME>_bcm23.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm23_c.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm23_atv.v	<IP_NAME>_bcm21_atv.v
Synchronizer 4: Gray Coded Synchronizer	<IP_NAME>_bcm24.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm24_atv.v	<IP_NAME>_bcm21_atv.v <IP_NAME>_bcm95_i.v
Synchronizer 5: Data Bus Synchronizer with Acknowledge	<IP_NAME>_bcm25.v	<IP_NAME>_bcm21.v <IP_NAME>_bcm23.v
	<IP_NAME>_bcm25_c.v	<IP_NAME>_bcm23_c.v
	<IP_NAME>_bcm93.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm25_atv.v	<IP_NAME>_bcm23_atv.v
Synchronizer 6: Data Bus Synchronizer without Acknowledge	<IP_NAME>_bcm26.v	<IP_NAME>_bcm22.v
	<IP_NAME>_bcm26_atv.v	<IP_NAME>_bcm22_atv.v

Table 1-1 BCM Synchronizers Used in Synthesizable IP (Continued)

Synchronizer Type and Number	BCM RTL File	Submodules
Synchronizer 7: Quasi-Synchronous Data Bus Synchronizer from Low Freq to High Freq	<IP_NAME>_bcm28.v	N/A
Synchronizer 8: Quasi-Synchronous Data Bus Synchronizer from High Frequency to Low Frequency	<IP_NAME>_bcm29.v	N/A
Synchronizer 9: Single Clock Data Bus Synchronizer	<IP_NAME>_bcm27.v	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm27_atv.v	<IP_NAME>_bcm21_atv.v
Synchronizer 10: Data Stream Synchronizer	<IP_NAME>_bcm39.v	<IP_NAME>_bcm21.v <IP_NAME>_bcm37.v
Synchronizer 11: Dual Clock FIFO Controller (simple version)	<IP_NAME>_bcm07*.v	<IP_NAME>_bcm05*.v <IP_NAME>_bcm21.v
	<IP_NAME>_bcm66*.v	<IP_NAME>_bcm07*.v <IP_NAME>_bcm58.v
	<IP_NAME>_bcm66*_atv.v	<IP_NAME>_bcm07*_atv.v <IP_NAME>_bcm58_atv.v
	<IP_NAME>_bcm87.v	<IP_NAME>_bcm85.v
Synchronizer 11: Dual Clock FIFO Controller (with prefetch cache)	<IP_NAME>_bcm38.v	<IP_NAME>_bcm08.v <IP_NAME>_bcm09.v <IP_NAME>_bcm21.v <IP_NAME>_bcm24.v <IP_NAME>_bcm37.v
	<IP_NAME>_bcm38_atv.v	<IP_NAME>_bcm22_atv.v <IP_NAME>_bcm95_i.v
	<IP_NAME>_bcm74.v	<IP_NAME>_bcm38.v <IP_NAME>_bcm58.v
	<IP_NAME>_bcm74_atv.v	<IP_NAME>_bcm38_atv.v <IP_NAME>_bcm58_atv.v
Synchronizer 12: Reset Sequence Synchronizer	<IP_NAME>_bcm37.v	<IP_NAME>_bcm21.v <IP_NAME>_bcm22.v
	<IP_NAME>_bcm37_atv.v	<IP_NAME>_bcm22_atv.v <IP_NAME>_bcm95_i.v
Synchronizer 13: Simple Reset Synchronizer	<IP_NAME>_bcm40	<IP_NAME>_bcm21.v
Synchronizer 14: Reset Synchronizer with Clock Gating	<IP_NAME>_bcm32_a	<IP_NAME>_bcm21.v
	<IP_NAME>_bcm32_c	<IP_NAME>_bcm21.v

Table 1-1 BCM Synchronizers Used in Synthesizable IP (Continued)

Synchronizer Type and Number	BCM RTL File	Submodules
Synchronizer 15: Compound Dual Clock FIFO	<IP_NAME>_bcm79	<IP_NAME>_bcm78.v <IP_NAME>_bcm66.v <IP_NAME>_bcm07.v
	<IP_NAME>_bcm79_atv	<IP_NAME>_bcm78_atv.v <IP_NAME>_bcm66_atv.v <IP_NAME>_bcm07_atv.v

1.2 Preserving Registers in BCMs During Synthesis

Some BCMs have special registers that should not be optimized away by synthesis tools. Table 1-2 provides details about these registers.

Table 1-2 Registers to Preserve

Register Type	Register Name	Implemented In
Triple module redundancy (TMR) registers	dw_so_reg*	<IP_NAME>_bcm95*
Synchronization	sample_meta	<IP_NAME>_bcm21 <IP_NAME>_bcm21_inj <IP_NAME>_bcm21_neo <IP_NAME>_bcm21_tgl <IP_NAME>_bcm99*
	sample_sync*	<IP_NAME>_bcm21 <IP_NAME>_bcm21_inj <IP_NAME>_bcm21_neo <IP_NAME>_bcm21_tgl <IP_NAME>_bcm98 <IP_NAME>_bcm99*
Duplicate feedback	*_nfb_cdc	<IP_NAME>_bcm22* <IP_NAME>_bcm23* <IP_NAME>_bcm39 <IP_NAME>_bcm93*

To set up a Synopsys synthesis tool so these registers are not optimized away, attach the “size_only” attribute to all registers in Table 1-2 using the following command:

```
set_size_only -all_instances [get_cells -hier *<reg name>*]
```

1. Apply the `set_size_only` command to all components in Table 1-2 that are in your design and to all their upper-level components.
2. If your design instantiates multiple BCM modules that contain more than one type of the registers in Table 1-2, use multiple commands to cover all the registers used.

In some cases, triple module redundancy (TMR) registers (dw_so_reg*) and duplicate feedback registers (*_nfb_cdc) are still removed during optimization despite having the "size_only" attribute. To strengthen the preservation of registers in these cases, use the following command:

```
set_register_merging [get_cells -hier *<reg_name>*] false
```

1.3 Synchronizer 1: Simple Multiple Register Synchronizer

Synchronizer 1 functionality is provided by:

- <IP_NAME>_bcm21*
- <IP_NAME>_bcm21_atv
- <IP_NAME>_bcm41*
- <IP_NAME>_bcm41_atv

Synchronizer 1 is a single clock data bus synchronizer for data that crosses asynchronous clock boundaries. Your synthesizable IP may have parameters that allow you to configure the number of synchronizing stages and the style of the first-stage capturing flip-flop (negative or positive edge-triggered).

Figure 1-1 Synchronizer 1: Missampling Model Type 0

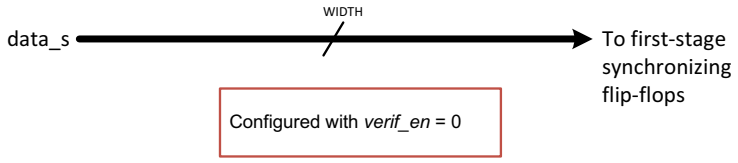


Figure 1-2 Synchronizer 1: Synchronization Type 1

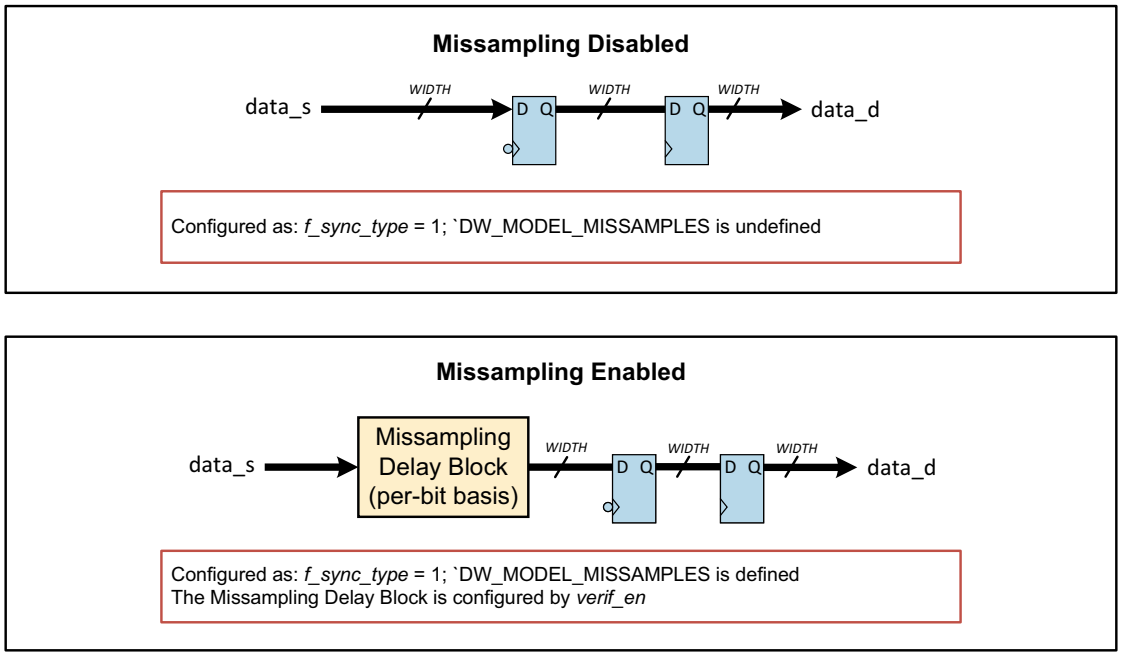


Figure 1-3 Synchronizer 1: Synchronization Type 2

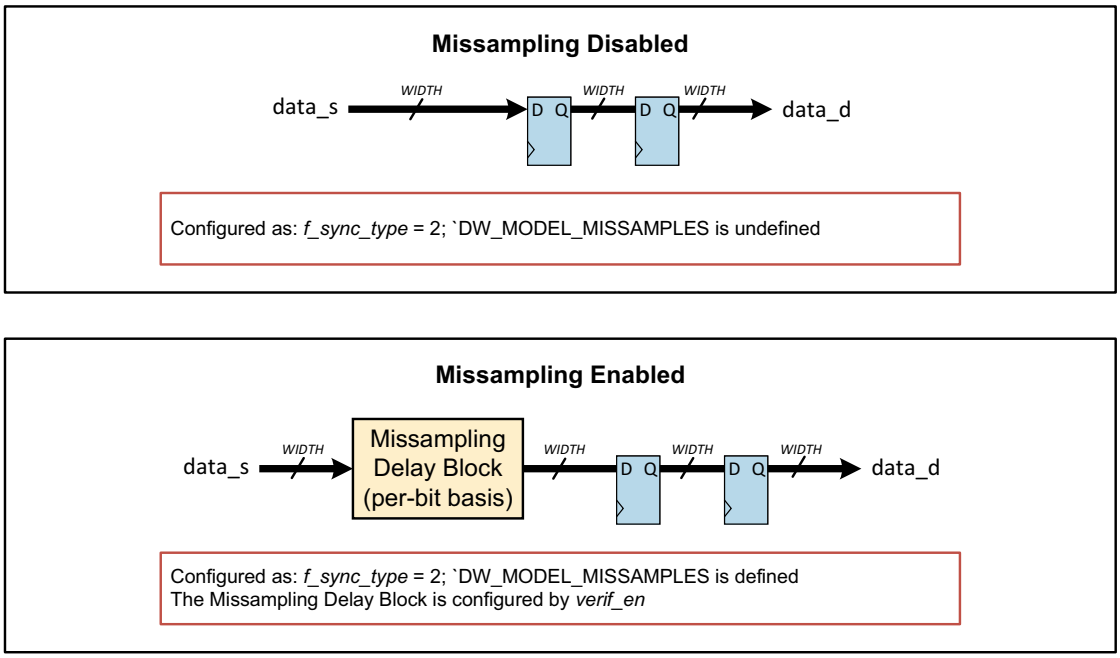


Figure 1-4 Synchronizer 1: Synchronization Type 3

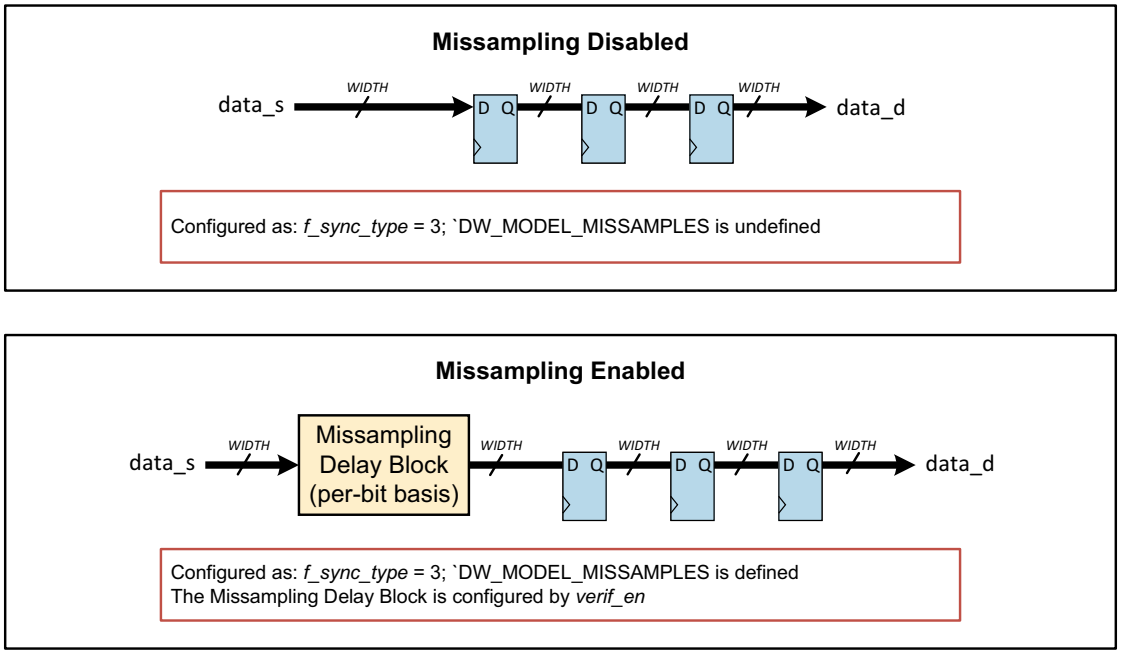


Figure 1-5 Synchronizer 1: Synchronization Type 4

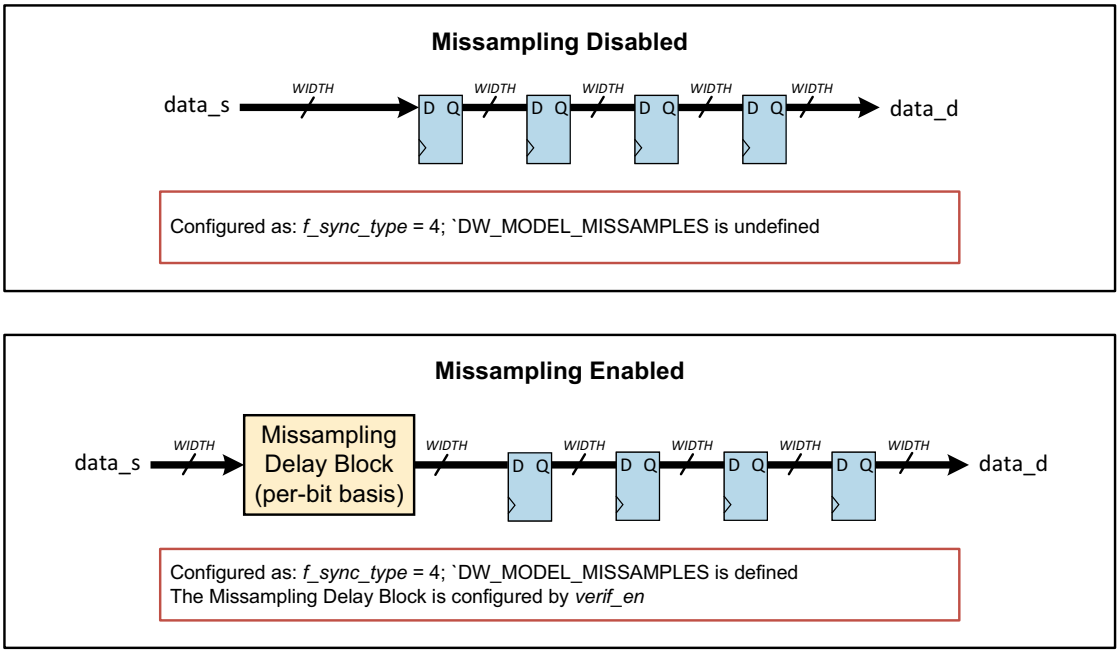


Figure 1-6 Synchronizer 1: Missampling Model Type 1

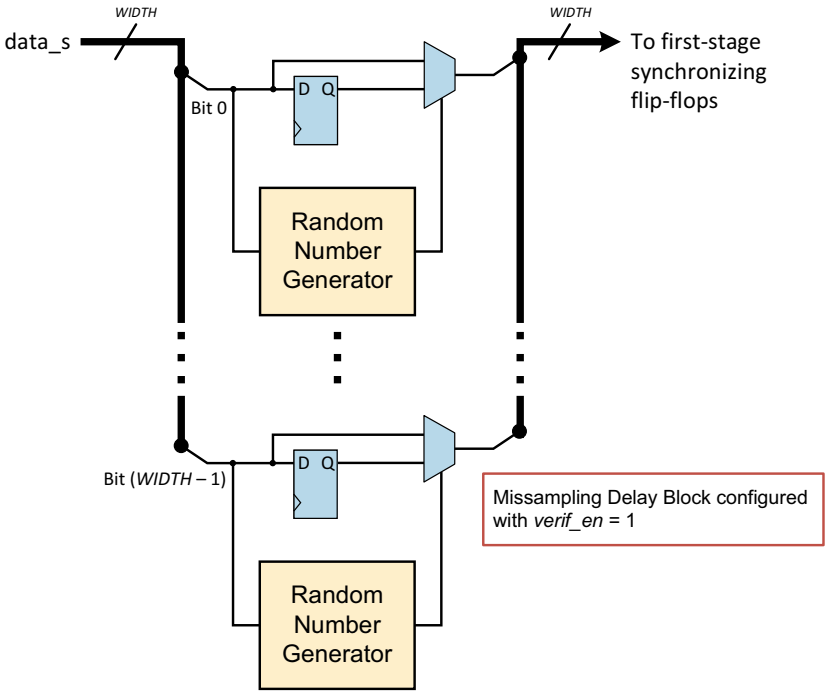
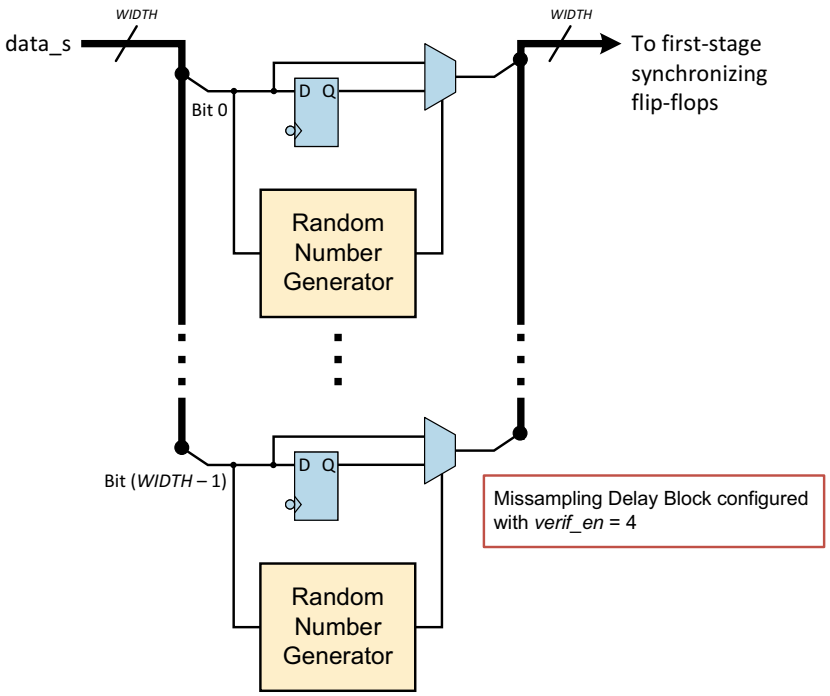


Figure 1-7 Synchronizer 1: Missampling Model Type 4



1.3.1 Synchronizer 1 for Automotive IP

Synchronizer 1 functionality for automotive IP is provided by:

<IP_NAME>_bcm21_atv

<IP_NAME>_bcm41_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip, and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

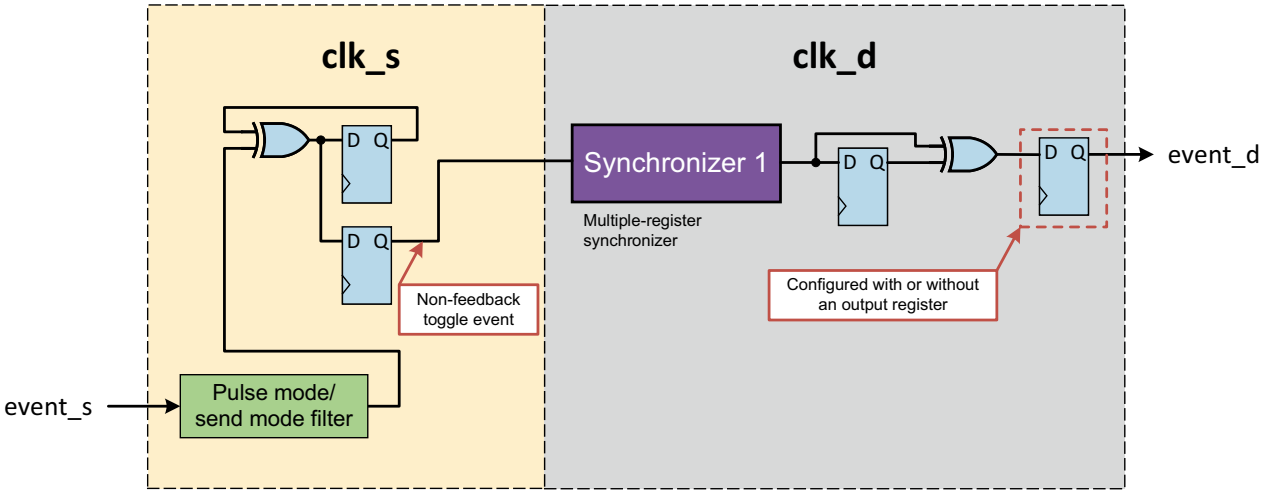
1.4 Synchronizer 2: Pulse Synchronizer

Synchronizer 2 functionality is provided by:

- <IP_NAME>_bcm22
- <IP_NAME>_bcm22_atv
- <IP_NAME>_bcm36

Synchronizer 2 is a dual clock pulse synchronizer for transmitting single clock cycle pulses between two clock domains. This method uses clock domain crossing techniques to safely transfer pulses between logic operating on different clocks.

Figure 1-8 Synchronizer 2 Block Diagram



1.4.1 Pulse Mode / Send Mode Filter for Synchronizer 2

The "pulse mode/send mode" filter is a logic block that can be configured to transform the event request input of a synchronizer in one of four ways:

- *SEND_MODE* or *PULSE_MODE* set to 0: Data sent or event generated for every clock cycle that the input is high (no filter).
- *SEND_MODE* or *PULSE_MODE* set to 1: Data sent or event generated for every clock cycle that the input is high but the input was low at the previous clock (sampled rising-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 2: Data sent or event generated for every clock cycle that the input is low but the input was high at the previous clock (sampled falling-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 3: Data sent or event generated for every clock cycle that the input is different from its value at the previous clock (sampled on change; that is, both rising- and falling-edge filter).

1.4.2 Synchronizer 2 for Automotive IP

Synchronizer 2 functionality for automotive IP is provided by:

- <IP_NAME>_bcm22_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip, and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

1.5 Synchronizer 3: Pulse Synchronizer with Acknowledge

Synchronizer 3 functionality is provided by:

- <IP_NAME>_bcm23
- <IP_NAME>_bcm23_c
- <IP_NAME>_bcm23_atv

Synchronizer 3 passes an event (represented by a single clock cycle pulse) from the source domain to the destination domain and passes an acknowledge. This synchronizer is used within other synchronizers to implement handshake mechanisms. It is constructed with feedback and an interlock (see "Pulse Mode / Send Mode Filter for Synchronizer 3" on page 21.

Possible configurations of Synchronizer 3 are shown in Figure 1-9 and Figure 1-10.

Figure 1-9 Synchronizer 3 Block Diagram (with Output Registers and Early ack Feedback)

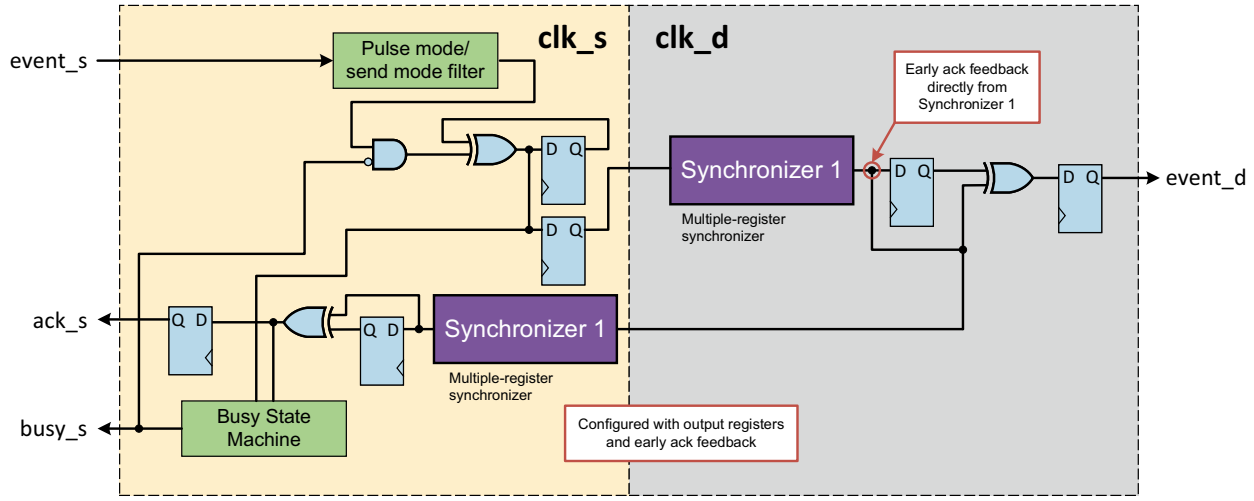
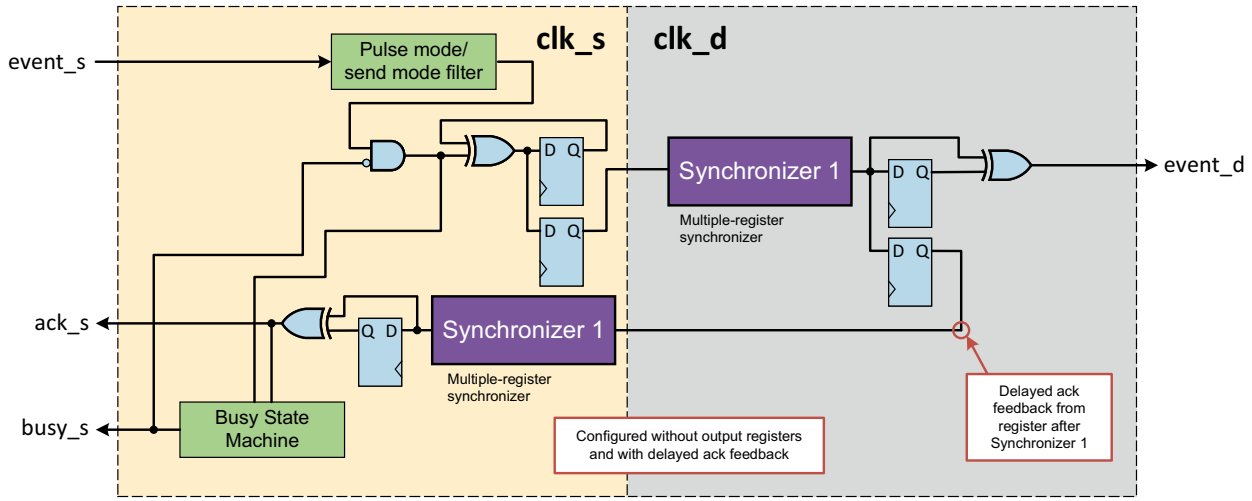


Figure 1-10 Synchronizer 3 Block Diagram (without Output Registers and with Delayed ack Feedback)



1.5.1 Pulse Mode / Send Mode Filter for Synchronizer 3

The "pulse mode/send mode" filter is a logic block that can be configured to transform the event request input of a synchronizer in one of four ways:

- *SEND_MODE* or *PULSE_MODE* set to 0: Data sent or event generated for every clock cycle that the input is high (no filter).
- *SEND_MODE* or *PULSE_MODE* set to 1: Data sent or event generated for every clock cycle that the input is high but the input was low at the previous clock (sampled rising-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 2: Data sent or event generated for every clock cycle that the input is low but the input was high at the previous clock (sampled falling-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 3: Data sent or event generated for every clock cycle that the input is different from its value at the previous clock (sampled on change; that is, both rising- and falling-edge filter).

1.5.2 Synchronizer 3 for Automotive IP

Synchronizer 3 functionality for automotive IP is provided by:

- <IP_NAME>_bcm23_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip, and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

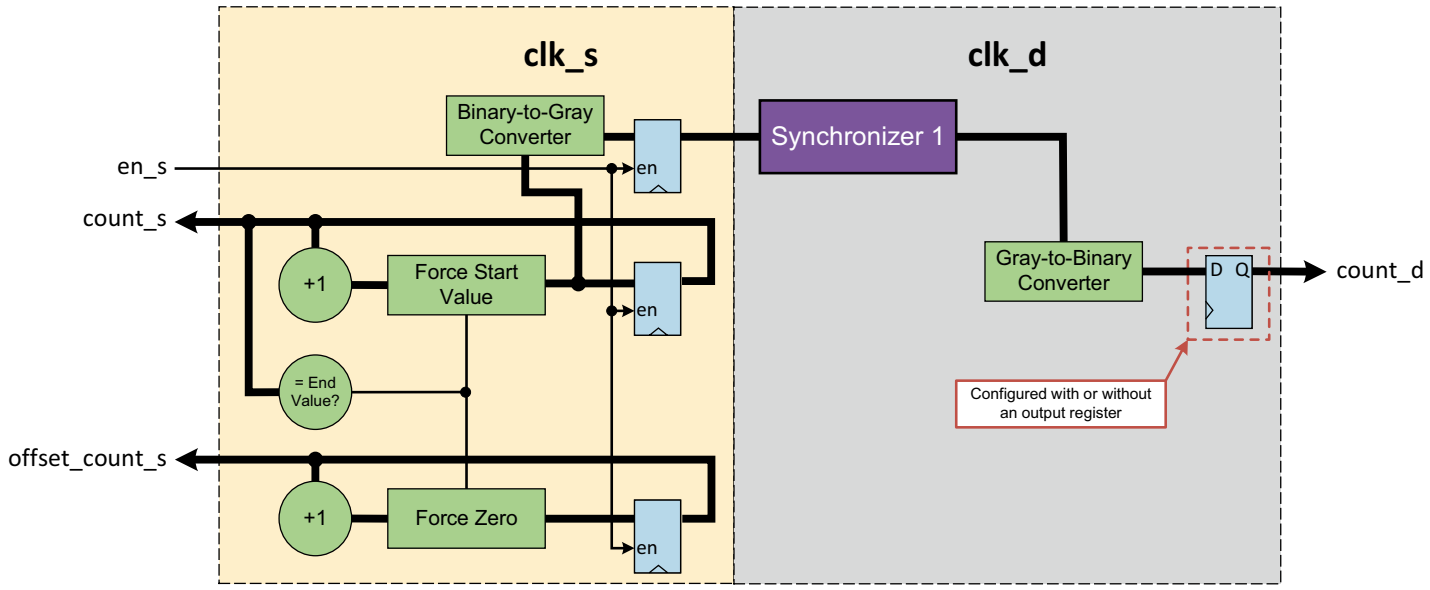
1.6 Synchronizer 4: Gray Coded Synchronizer

Synchronizer 4 functionality is provided by:

- <IP_NAME>_bcm24
- <IP_NAME>_bcm24_atv

As shown in Figure 1-11, Synchronizer 4 includes a binary counter in the source domain whose value is recoded as a binary-reflected Gray code. The recoded count value is synchronized to the destination domain and then decoded from binary-reflected Gray back to binary. The binary count value and an offset version (which is only useful for non-integer power-of-two sequences) are both available in the `clk_s` domain as `count_s` and `offset_count_s`. This synchronizer is used within other synchronizers to pass incrementing pointers across domains.

Figure 1-11 Synchronizer 4 Block Diagram



1.6.1 Synchronizer 4 for Automotive IP

Synchronizer 4 functionality for automotive IP is provided by:

- <IP_NAME>_bcm24_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

1.7 Synchronizer 5: Data Bus Synchronizer with Acknowledge

Synchronizer 5 functionality is provided by:

- <IP_NAME>_bcm25
- <IP_NAME>_bcm25_atv
- <IP_NAME>_bcm25_c
- <IP_NAME>_bcm93*

Synchronizer 5 passes data values from the source domain to the destination domain and passes an acknowledge back. It is constructed with feedback and an interlock (for more, see "Pulse Mode / Send Mode Filter" on page 24).

Also, the <IP_NAME>_bcm25, <IP_NAME>_bcm25_atv modules can provide a pending data buffer register to save pending data given during a data transfer, which is shown in Figure 1-12.

Figure 1-12 Synchronizer 5 Block Diagram

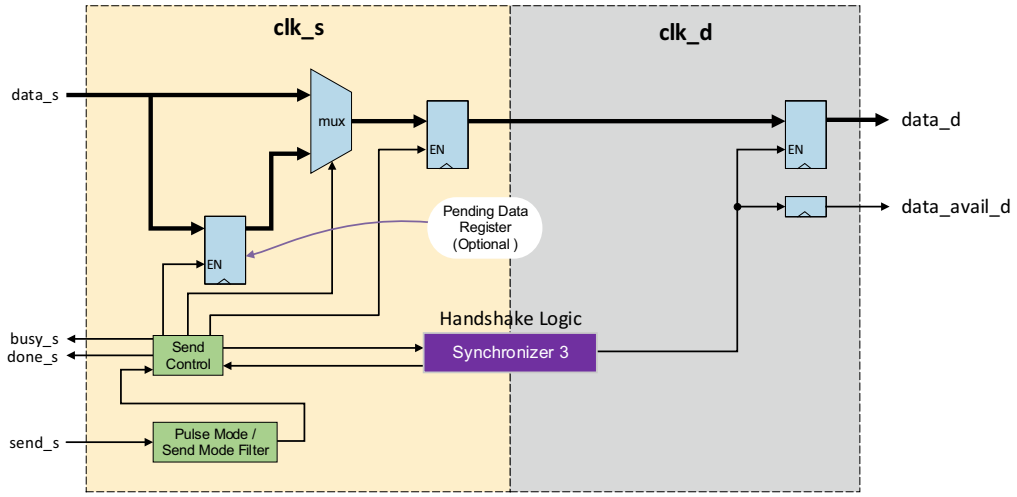
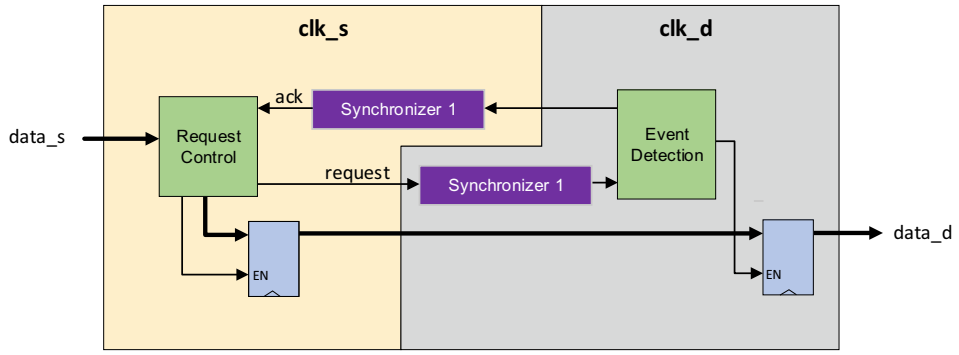


Figure 1-13 is an additional block diagram of <IP_NAME>_bcm93, which shows that changes to data at the source automatically generate trigger (toggle) requests.

Figure 1-13 Synchronizer 5 Block Diagram for <IP_NAME>_bcm93 Showing Auto-Generated Toggle Requests



1.7.1 Pulse Mode / Send Mode Filter

The <IP_NAME>_bcm25 and <IP_NAME>_bcm25_atv modules provide a "pulse mode/send mode" filter (shown in Figure 1-12 on page 23) that can be configured to transform the event request input of a synchronizer in one of four ways:

- *SEND_MODE* or *PULSE_MODE* set to 0: Data sent or event generated for every clock cycle that the input is high (no filter).
- *SEND_MODE* or *PULSE_MODE* set to 1: Data sent or event generated for every clock cycle that the input is high but the input was low at the previous clock (sampled rising-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 2: Data sent or event generated for every clock cycle that the input is low but the input was high at the previous clock (sampled falling-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 3: Data sent or event generated for every clock cycle that the input is different from its value at the previous clock (sampled on change; that is, both rising- and falling-edge filter).

1.7.2 Synchronizer 5 for Automotive IP

Synchronizer 5 functionality for automotive IP is provided by:

- <IP_NAME>_bcm25_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

1.8 Synchronizer 6: Data Bus Synchronizer without Acknowledge

Synchronizer 6 functionality is provided by:

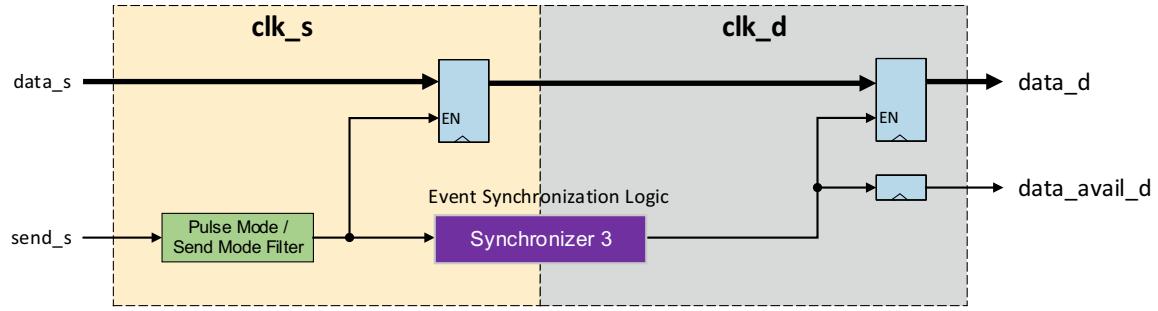
- <IP_NAME>_bcm26
- <IP_NAME>_bcm26_atv

Synchronizer 6 passes data values from the source domain to the destination domain. Full feedback handshake is not used so there is no busy or done status in the source domain.

This synchronizer is useful in situations when the destination domain is known to always have at least a 3X faster clock rate than the source domain (so that every transaction started by the source domain is guaranteed to be completed in the destination domain before another one can be started in the source domain).

Figure 1-14 shows a block diagram of Synchronizer 6. The "pulse mode/send mode" block in Figure 1-14 is described in "Pulse Mode / Send Mode Filter".

Figure 1-14 Synchronizer 6 Block Diagram



1.8.1 Pulse Mode / Send Mode Filter

The "pulse mode/send mode" filter is a logic block that can be configured to transform the event request input of a synchronizer in one of four ways:

- *SEND_MODE* or *PULSE_MODE* set to 0: Data sent or event generated for every clock cycle that the input is high (no filter).
- *SEND_MODE* or *PULSE_MODE* set to 1: Data sent or event generated for every clock cycle that the input is high but the input was low at the previous clock (sampled rising-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 2: Data sent or event generated for every clock cycle that the input is low but the input was high at the previous clock (sampled falling-edge filter)
- *SEND_MODE* or *PULSE_MODE* set to 3: Data sent or event generated for every clock cycle that the input is different from its value at the previous clock (sampled on change; that is, both rising- and falling-edge filter).

1.8.2 Synchronizer 6 for Automotive IP

Synchronizer 6 functionality for automotive IP is provided by:

- <IP_NAME>_bcm26_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

1.9 Synchronizer 7: Quasi-Synchronous Data Bus Synchronizer from Low Freq to High Freq

Synchronizer 7 functionality is provided by:

- <IP_NAME>_bcm28

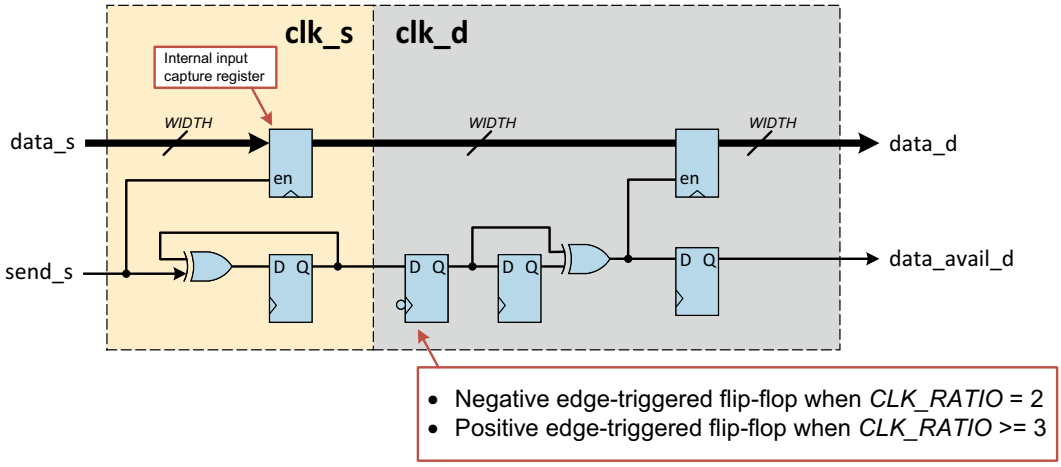
Synchronizer 7 passes data values from a low frequency domain to a high frequency domain. The two domains are synchronous with respect to each other, with the low frequency clock expected to be a derivative of the high frequency clock. Therefore, the derived clock (slow clock) could contain jitter and skew uncertainties with respect to the originating clock source (the fast clock).

The *CLK_RATIO* parameter (integer value) is determined by the high frequency divided by the low frequency (or the slower clock period divided by the faster clock period). Full feedback handshake is not used.

Note that, when the *CLK_RATIO* is 2, a negative edge-triggered flip-flop is implemented between the two clock boundaries.

Figure 1-15 shows a block diagram of <IP_NAME>_bcm28.

Figure 1-15 Synchronizer 7 Block Diagram



Examples of timing for Synchronizer 7 (showing clock ratios of 2 and 3) are provided in Figures 1-16 and Figure 1-17 on page 27.

Figure 1-16 Synchronizer 7 Timing example #1: *CLK_RATIO* = 2

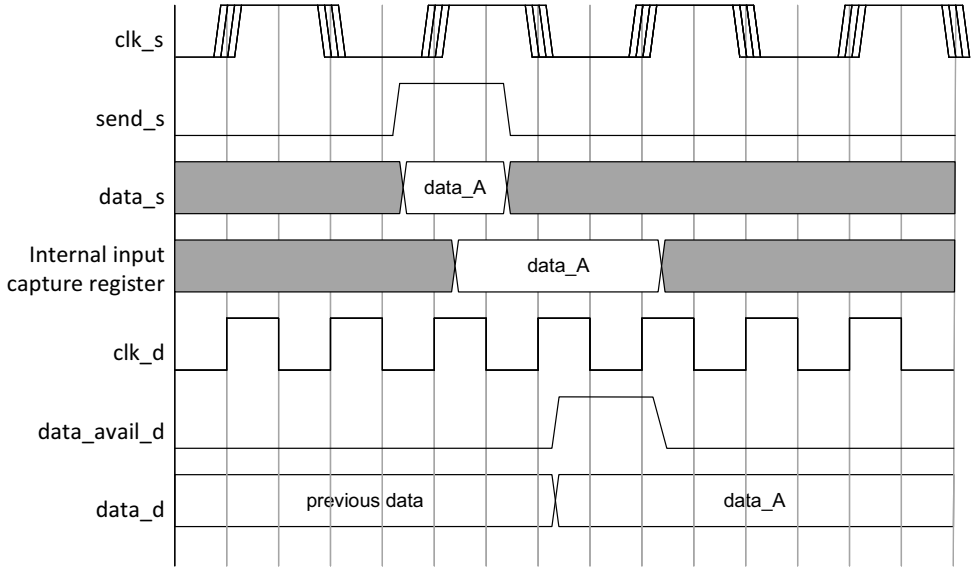
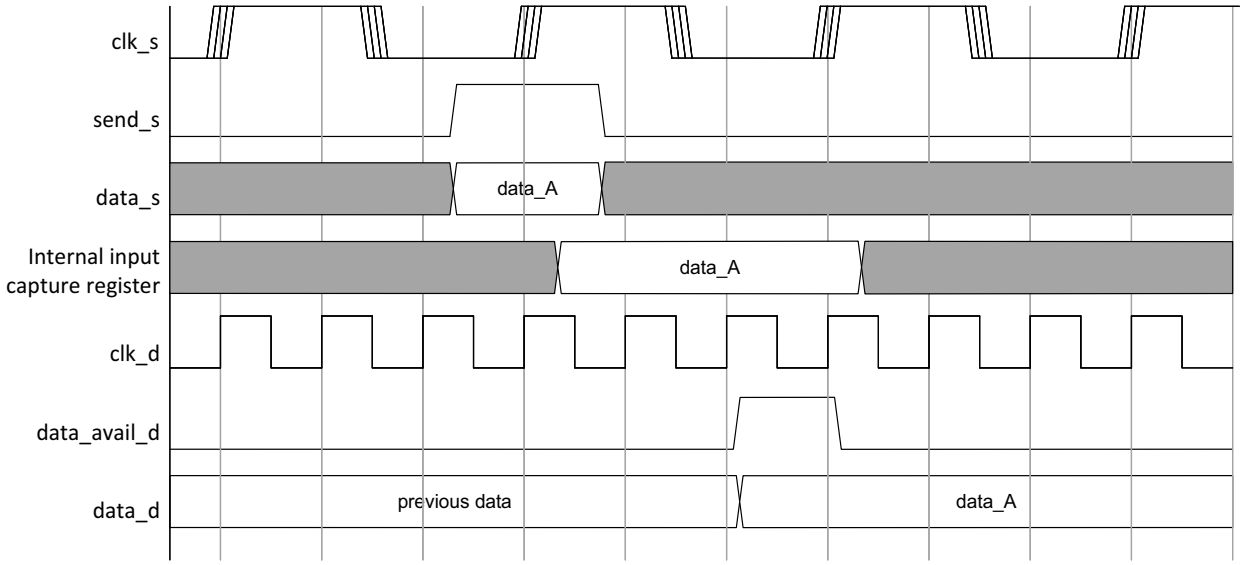


Figure 1-17 Synchronizer 7 Timing example #2: *CLK_RATIO* = 3



1.10 Synchronizer 8: Quasi-Synchronous Data Bus Synchronizer from High Frequency to Low Frequency

Synchronizer 8 functionality is provided by:

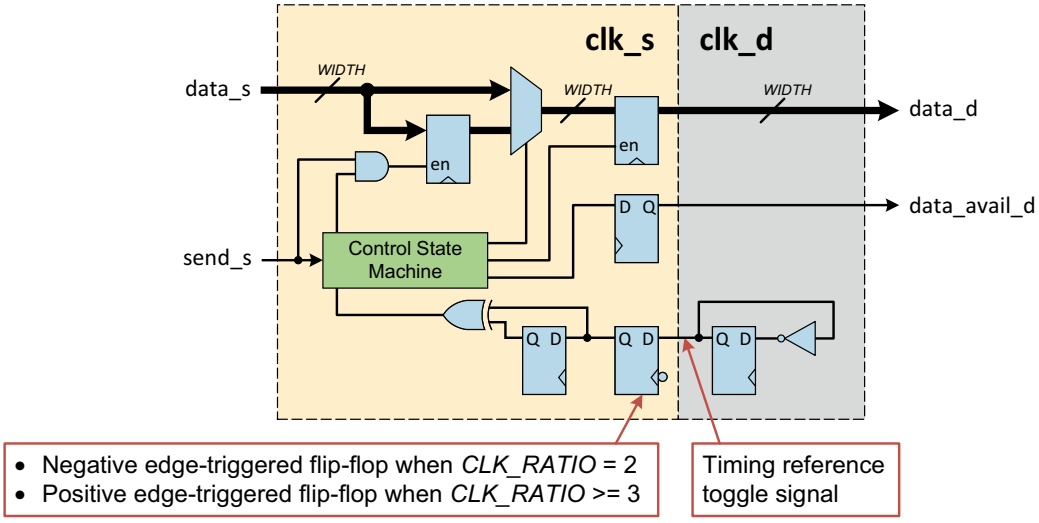
- <IP_NAME>_bcm29

As shown in Figure 1-18, Synchronizer 8 passes data values from a high frequency domain to a low frequency domain. The two domains are synchronous with respect to each other, with the low frequency clock expected to be a derivative of the high frequency clock. Therefore, the derived clock (slow clock) could contain jitter and skew uncertainties with respective to the originating clock source (the fast clock).

The *CLK_RATIO* parameter (integer value) is determined by the high frequency divided by low frequency (or the slower clock's period divided by the faster clock's period). Full feedback handshake is not used.

Note that, when the *CLK_RATIO* is 2, a negative edge-triggered flip-flop is implemented between the two clock boundaries.

Figure 1-18 Synchronizer 8 Block Diagram



Examples of timing for Synchronizer 8 are provided in Figure 1-19 on page 29 through Figure 1-22 on page 30.

Figure 1-19 shows *CLK_RATIO* = 2 and data arrives one `clk_s` cycle before the rising edge of `clk_d`; `data_s` is clocked directly to `data_d`.

Figure 1-19 Synchronizer 8 Timing Example #1: *CLK_RATIO* = 2

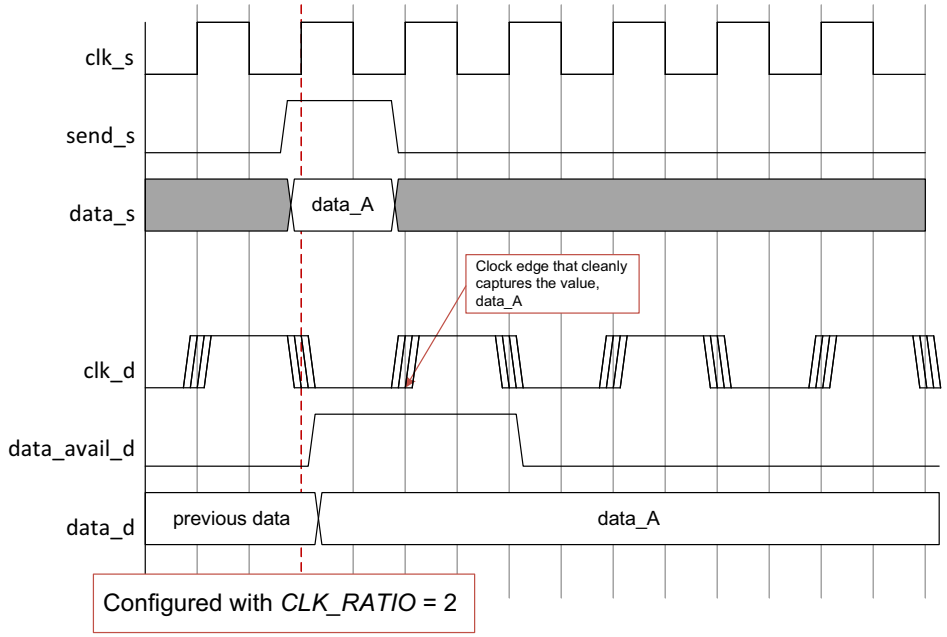


Figure 1-20 shows *CLK_RATIO* = 2 and data arrives concurrent with the rising edge of `clk_d`; `data_s` is clocked directly to `data_d` output register.

Figure 1-20 Synchronizer 8 Timing Example #2: *CLK_RATIO* = 2

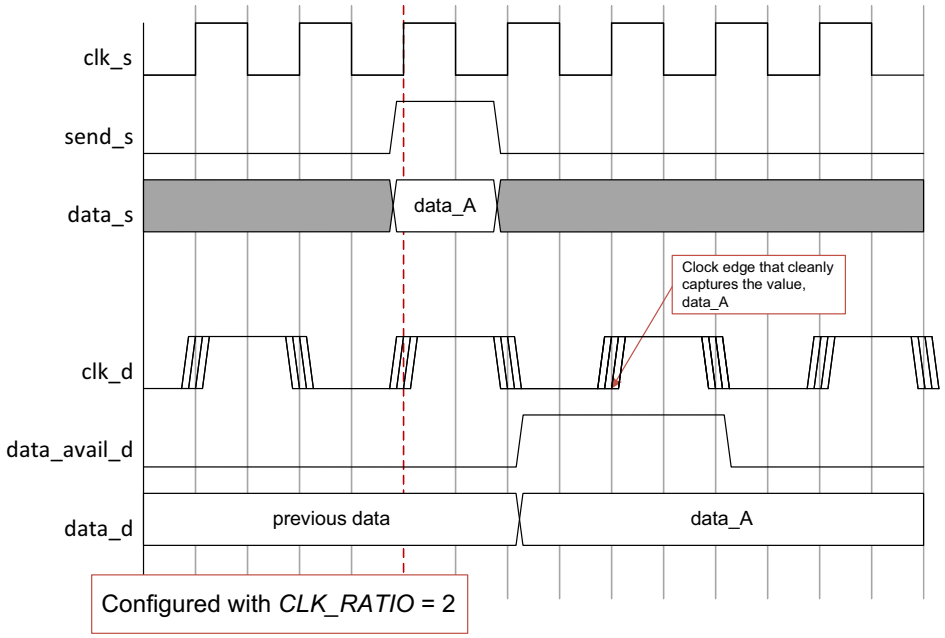


Figure 1-21 shows *CLK_RATIO* = 3 and data arrives one `clk_s` cycle before the rising edge of `clk_d`; `data_s` is clocked directly to `data_d`.

Figure 1-21 Synchronizer 8 Timing Example #3: *CLK_RATIO* = 3

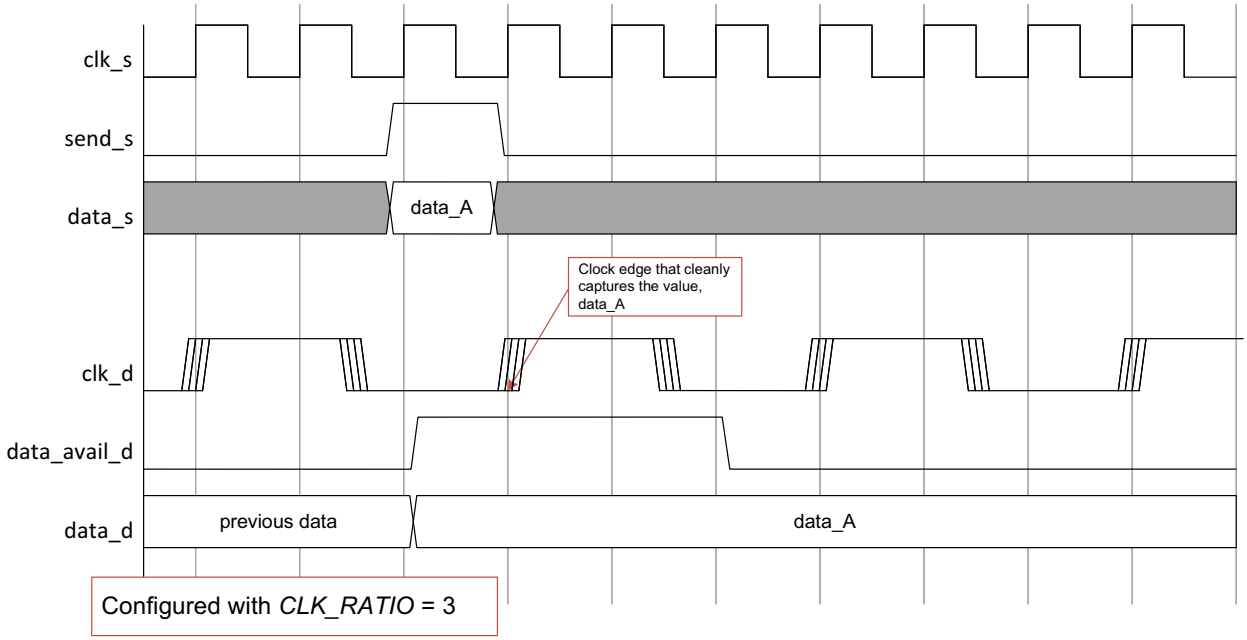
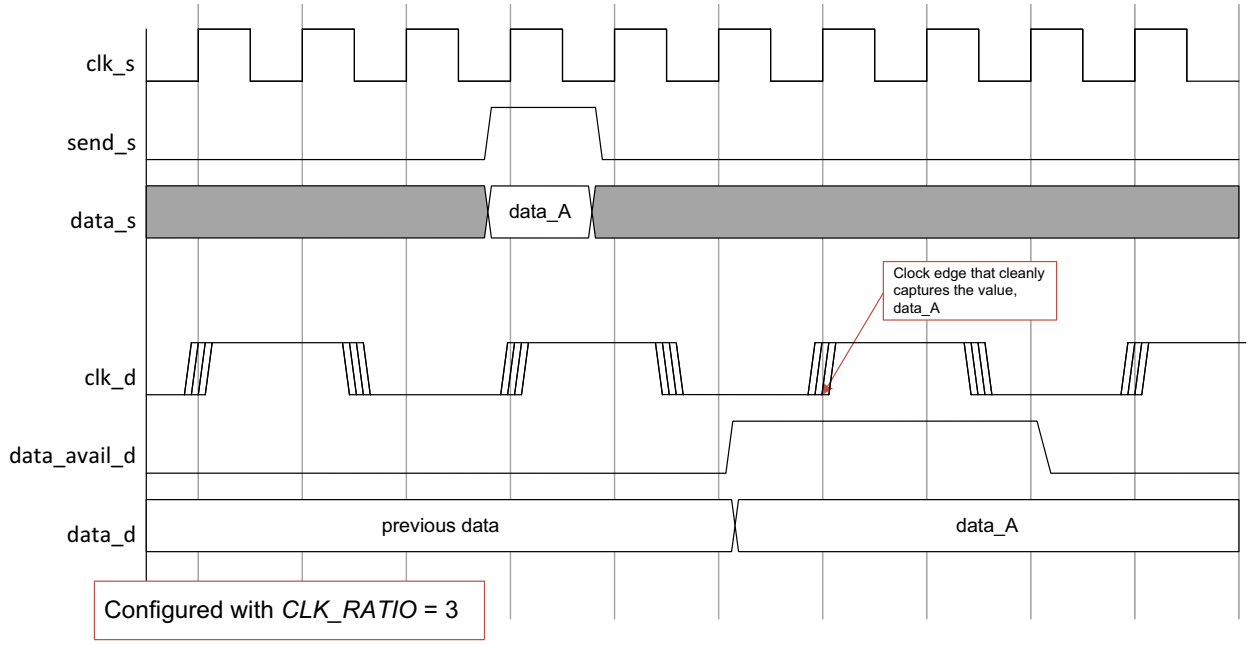


Figure 1-22 shows *CLK_RATIO* = 3 and data arrives concurrent with the rising edge of `clk_d`; `data_s` is clocked directly into temp register before being clocked into `data_d` output register.

Figure 1-22 Synchronizer 8 Timing example #4: *CLK_RATIO* = 3



1.11 Synchronizer 9: Single Clock Data Bus Synchronizer

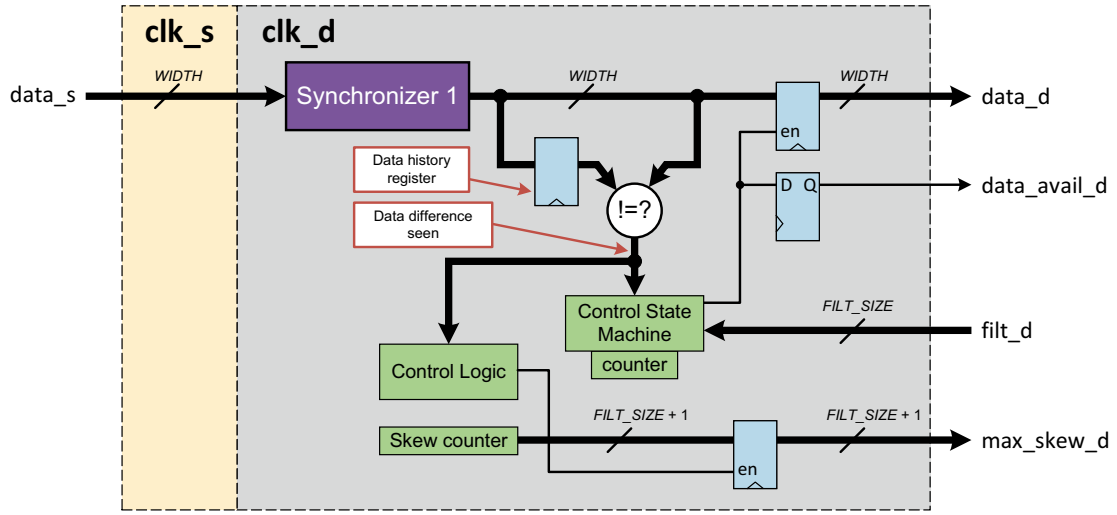
Synchronizer 9 functionality is provided by:

- <IP_NAME>_bcm27
- <IP_NAME>_bcm27_atv

As shown in Figure 1-23, Synchronizer 9 synchronizes a data bus to a clock domain when only the data bus is available but not the source clock. The bus is multiple-register synchronized (which is configurable up to thee stages of synchronization) and then monitored for changes. Once a change is detected, the module looks for a specified number of clocks of stability in the data bus before passing the synchronized data bus value out. The number of cycles required to declare the bus stable is specified by the input port `filt_d` and has a range that is determined by the parameter *FILT_SIZE*.

This synchronizer is especially well suited for synchronizing a bus from off-chip, where the input skew between bits of the bus may have a large variance due to board-level wiring and the skew also changes from board design to board design.

Figure 1-23 Synchronizer 9 Block Diagram



1.11.1 Synchronizer 9 for Automotive IP

Synchronizer 9 functionality for automotive IP is provided by:

- <IP_NAME>_bcm27_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

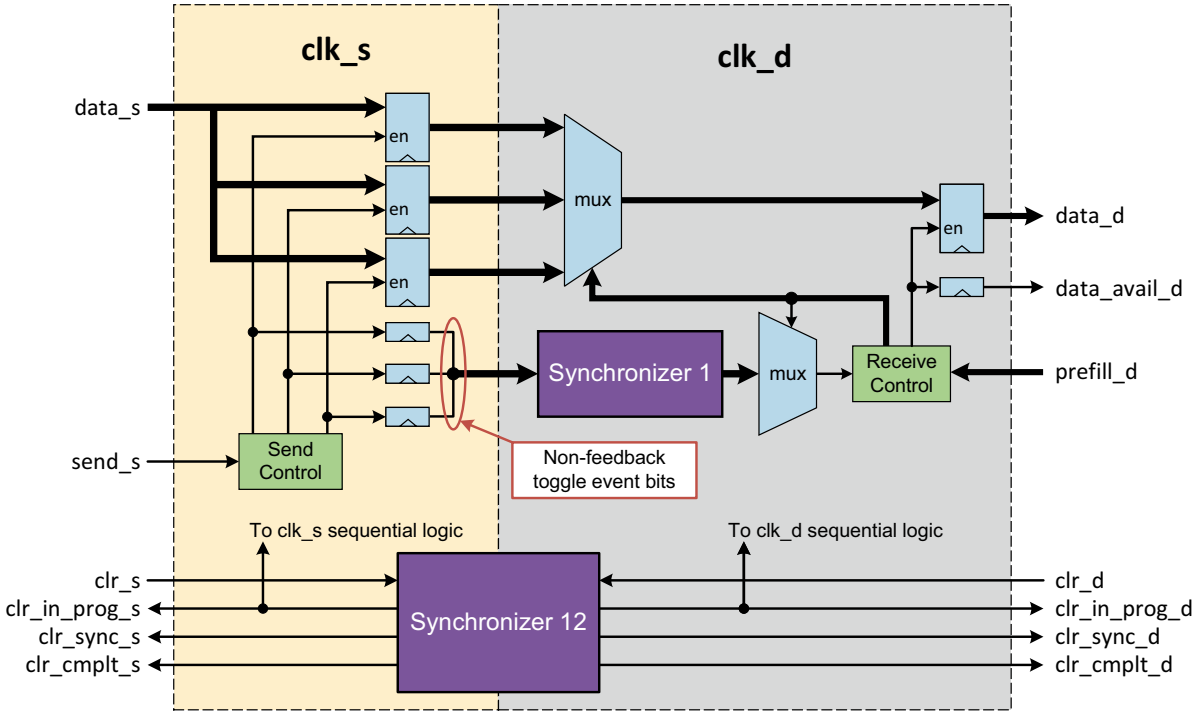
1.12 Synchronizer 10: Data Stream Synchronizer

Synchronizer 10 functionality is provided by:

- <IP_NAME>_bcm39

As shown in Figure 1-18, Synchronizer 10 synchronizes an incoming data stream from a source domain to a destination domain with a minimum amount of latency. As long as the aggregate data rate from the source domain does not exceed the destination flow rate, a wide variety of disparate clock rates can be used. Using the parameter *PREFILL_LVL*, the data stream FIFO can be directed to prefill the FIFO to a predetermined level so as to be used as an elasticity buffer.

Figure 1-24 Synchronizer 10 Block Diagram



1.13 Synchronizer 11: Dual Clock FIFO Controller

Synchronizer 11 functionality is provided by:

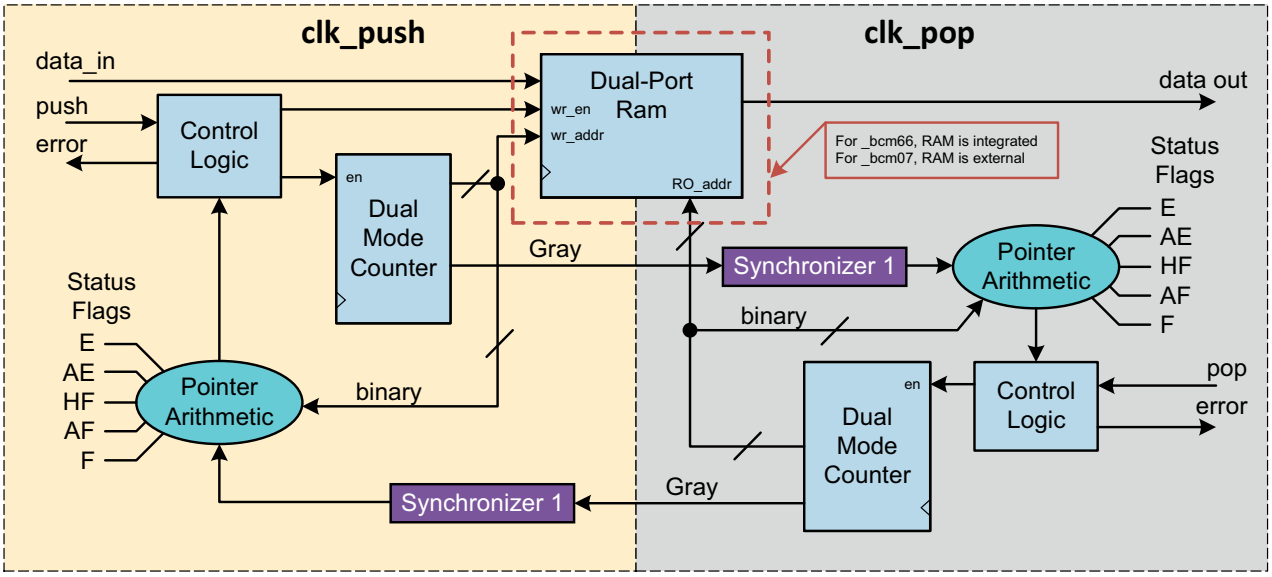
- <IP_NAME>_bcm07*
- <IP_NAME>_bcm87
- <IP_NAME>_bcm66*
- <IP_NAME>_bcm66*_atv
- <IP_NAME>_bcm38
- <IP_NAME>_bcm38_atv
- <IP_NAME>_bcm74
- <IP_NAME>_bcm74_atv

Synchronizer 11 implements the functions required to implement a FIFO once connected to a RAM. The push and pop interfaces are in different clock domains. Gray-coded pointers are used to pass information between domains.

1.13.1 Synchronizer 11 as <IP_NAME>_bcm07, <IP_NAME>_bcm87, or <IP_NAME>_bcm66

Figure 1-25 shows a block diagram of Synchronizer 11 when it consists of <IP_NAME>_bcm07, <IP_NAME>_bcm87, or <IP_NAME>_bcm66.

Figure 1-25 Synchronizer 11 Block Diagram Consisting of <IP_NAME>_bcm07, <IP_NAME>_bcm87, or <IP_NAME>_bcm66

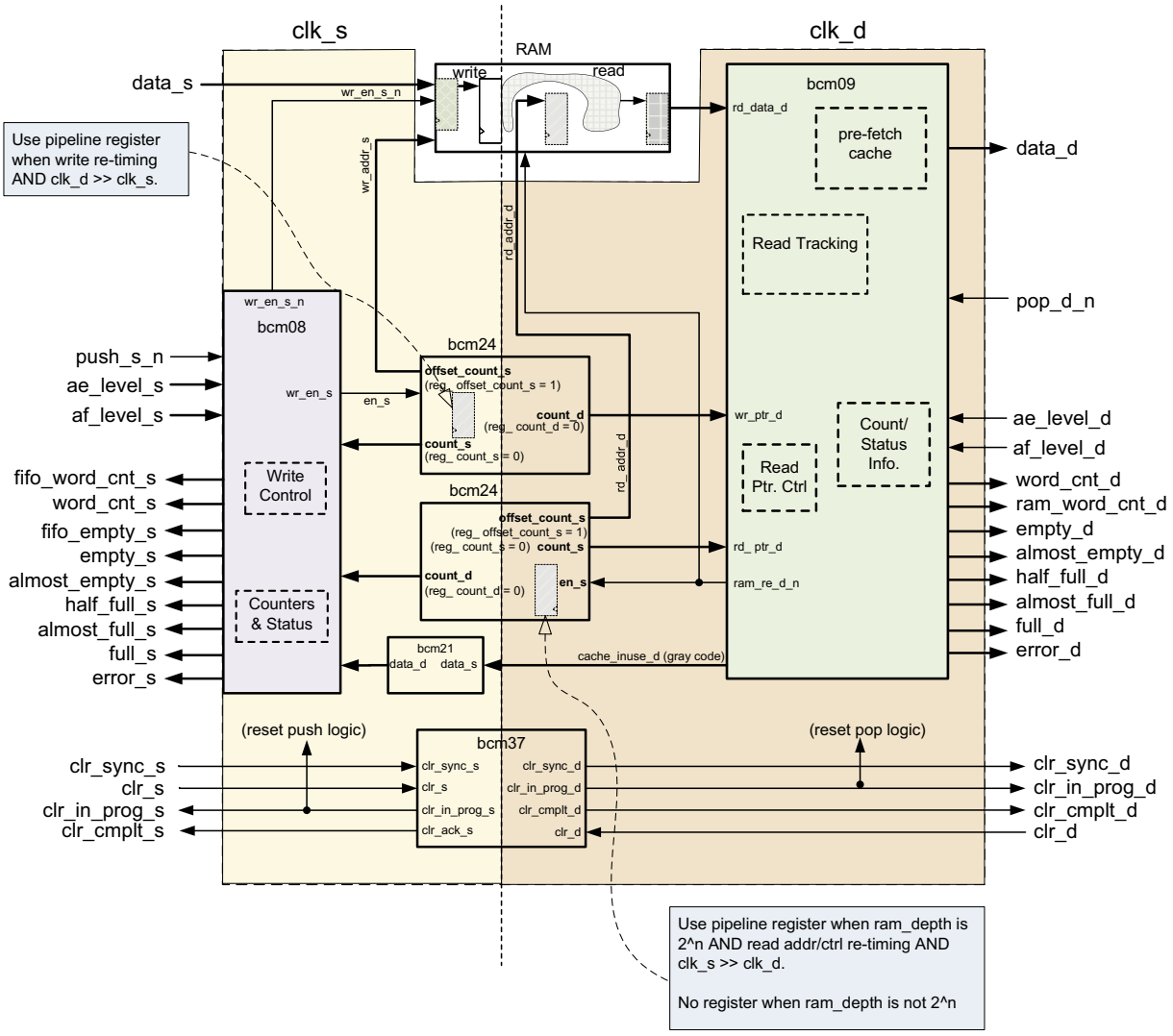


The design shown in Figure 1-25 embodies a simple form of Synchronizer 11. The <IP_NAME>_bcm07 FIFO controller is used to control the read and write operations as instructed by push and pop controls. In this design, the FIFO data in and data out buses are directly connected to the `data_in` and at `data_out` ports of the <IP_NAME>_bcm57 RAM. Other possible RAM modules are discussed in "RAMs for Synchronizer 11" on page 36.

1.13.2 Synchronizer 11 as <IP_NAME>_bcm38 or <IP_NAME>_bcm74

Figure 1-26 shows a block diagram of Synchronizer 11 when it consists of <IP_NAME>_bcm38 or <IP_NAME>_bcm74.

Figure 1-26 Synchronizer 11 Block Diagram Consisting of <IP_NAME>_bcm38 or <IP_NAME>_bcm74



For the design in Figure 1-26, the <IP_NAME>_bcm38 design implements a “read prefetch caching” FIFO controller. It can be configured to operate with RAM that may have retiming registers at the write input, read input and read data output. To maintain full bursting of data, the pop logic prefetches data. The more retiming in the RAM read path, the more caching is required. Because the prefetch logic reads FIFO data from RAM and caches it, the read data from the RAM passes through the FIFO controller. The <IP_NAME>_bcm38 FIFO controller also includes a synchronous FIFO clearing mechanism through an instance of <IP_NAME>_bcm37 (reset synchronizer) which controls a multiphase 2-clock initialization.

The <IP_NAME>_bcm74 dual clock FIFO design uses the <IP_NAME>_bcm07 module along with the <IP_NAME>_bcm57 single clock RAM. (If you need retiming registers, use <IP_NAME>_bcm58 two-clock RAM instead). Possible RAM modules are discussed in “RAMs for Synchronizer 11” on page 36.

1.13.2.1 Source Domain Interface Module (<IP_NAME>_bcm08)

As shown in Figure 1-26 on page 34, the <IP_NAME>_bcm08 module orchestrates the pushing operation (source domain) into the FIFO controller and manages the status in the source (push) domain.

When `push_s_n` is asserted (and `push_full` is not asserted), a write operation is initiated to RAM with `wr_en_s_n` asserted. Additionally, the `wr_en_s` going to the <IP_NAME>_bcm24 is asserted, which increments `wr_addr_s`.

The <IP_NAME>_bcm08 updates status flags and also contains counters that give the state of the FIFO controller in the context of the source domain based on inputs `wr_ptr_s`, `rd_ptr_s`, and `cache_inuse_s`.

1.13.2.2 Destination Domain Interface Module (<IP_NAME>_bcm09)

As shown in Figure 1-26 on page 34, the <IP_NAME>_bcm09 module provides the popping interface that contains output buffering (prefetching cache), with the number of pipeline stages determined by the *MEM_MODE* parameter. When the cache is not fully populated with valid data and the RAM is detected as having valid entries, data is automatically prefetched into the cache to provide immediate data availability at pop requests no matter which mode of read port the RAM uses (asynchronous versus one-deep synchronous versus two-deep synchronous). An extra latency applies, not only to when the first data word arrives at the pop interface, but also to when FIFO "fullness" information is delivered to `word_cnt_d` and the pop interface status flag ports. At a minimum, there is always at least one buffering stage in the cache that is seen at the pop interface. However, only the first word of a burst of words incurs a one `clk_d` cycle latency before being read from the RAM and presented to the pop interface. Table 1-3 lists the number of prefetching stages of the cache used based on the *MEM_MODE* parameter value.

Table 1-3 Pop Interface Cache Sizes

***MEM_MODE* Values**	**Number of Caching Stages**
0 or 4	1
1, 2, 5, or 6	2
3 or 7	3

1.13.2.3 Reading from the Memory

In Figure 1-26 on page 34, the read port of the RAM must be either asynchronous or synchronous with its own clock (unique from the write port clock). All read data from RAM is first loaded into the prefetching cache. When `ram_re_d_n` is asserted, it initiates a read operation from RAM with the `rd_addr_d` (read address) output value from <IP_NAME>_bcm24. Reading the RAM is initiated in either of the following ways:

- The RAM is not empty and the cache is not full.
- `pop_d_n` is asserted while the cache contains at least one valid data entry (and the RAM is not empty).

The <IP_NAME>_bcm09 module also updates status flags and contains counters that give the state of the FIFO controller in the context of the destination domain based on inputs `wr_ptr_d`, `rd_ptr_d`, and the internal cache state.

1.13.3 RAMs for Synchronizer 11

Figure 1-27 and Figure 1-28 on page 37 are block diagrams depicting the <IP_NAME>_bcm57 and <IP_NAME>_bcm58 modules that are RAMs commonly used with FIFO controllers to construct FIFOs. The <IP_NAME>_bcm57 module is usually used with the <IP_NAME>_bcm07 FIFO controller (with <IP_NAME>_bcm57 configured without any retiming registers) and <IP_NAME>_bcm58 is usually used with <IP_NAME>_bcm38 controller.

Figure 1-27 <IP_NAME>_bcm57 Single Clock RAM

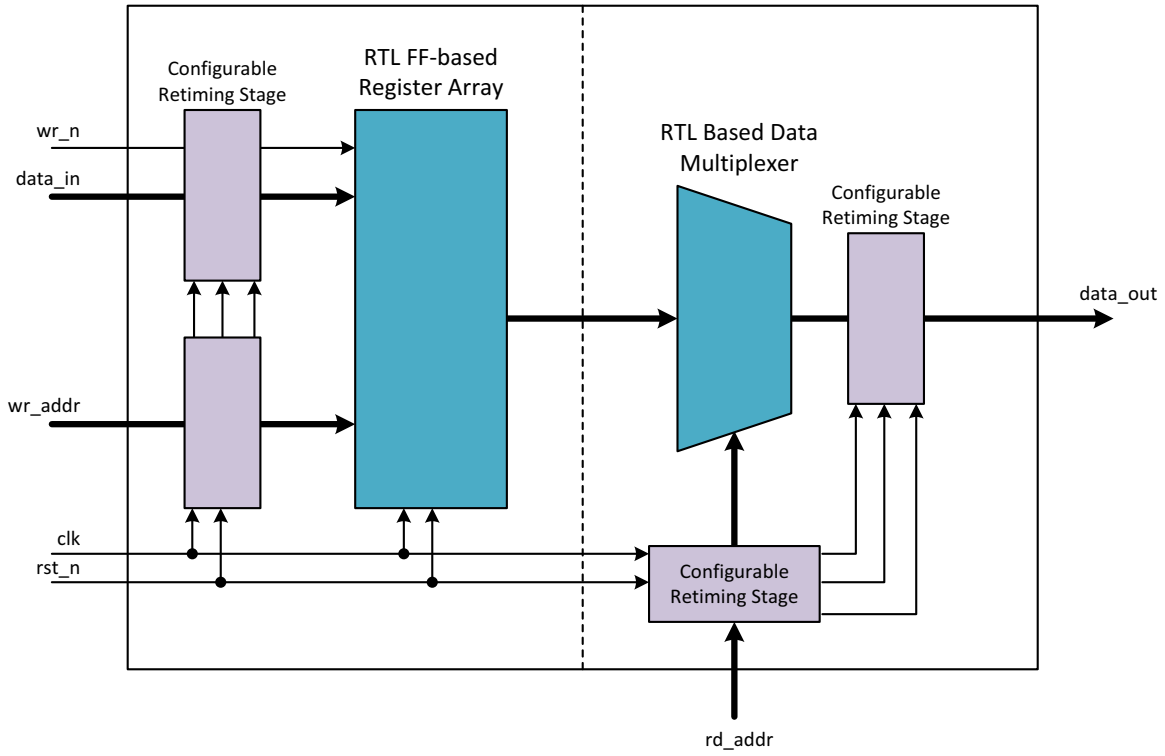
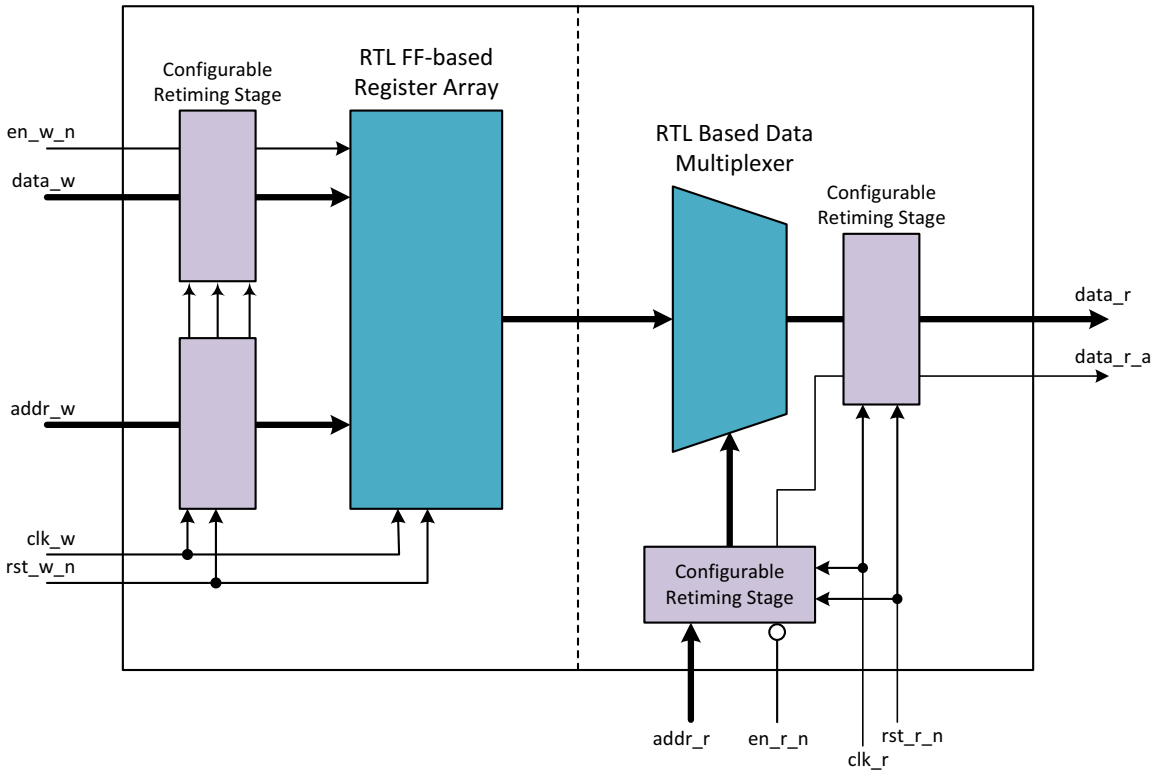


Figure 1-28 <IP_NAME>_bcm58 Two-Clock RAM



1.13.4 Synchronizer 11 for Automotive IP

Synchronizer 11 functionality for automotive IP is provided by:

- <IP_NAME>_bcm66_atv
- <IP_NAME>_bcm38_atv
- <IP_NAME>_bcm74_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

1.14 Synchronizer 12: Reset Sequence Synchronizer

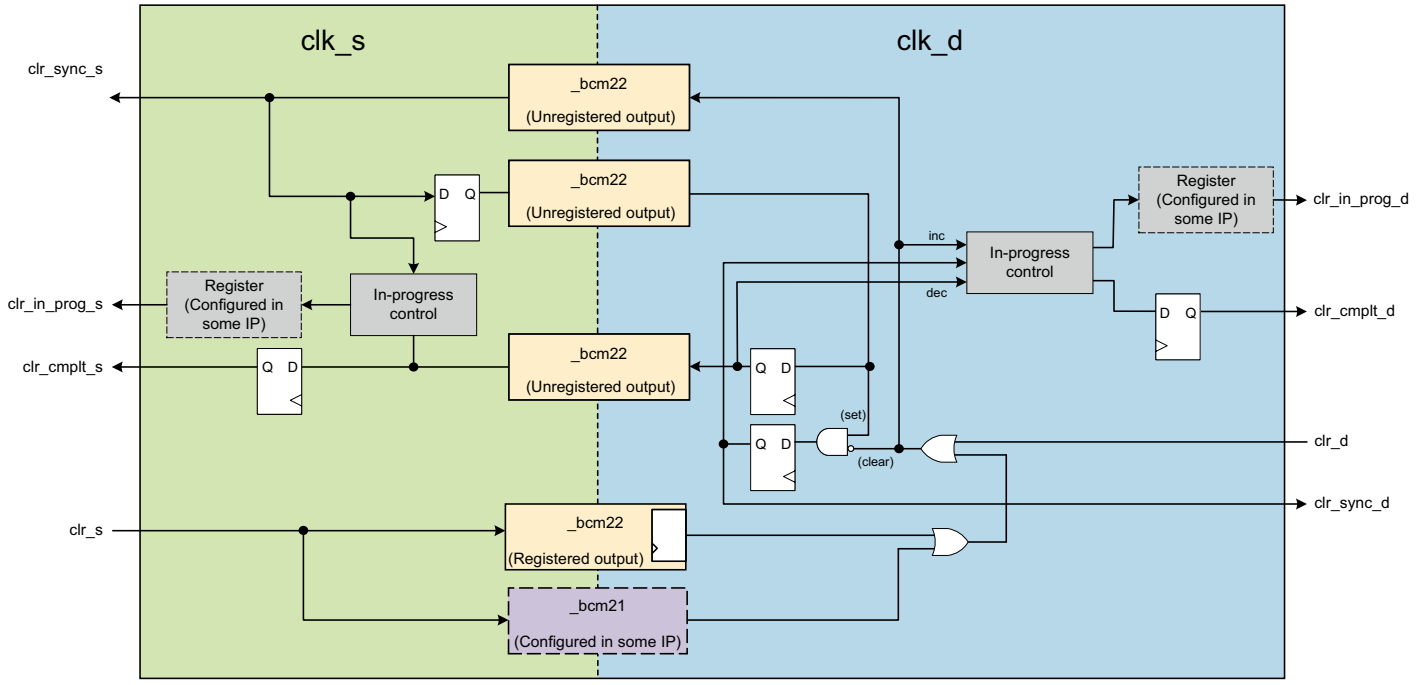
Synchronizer 12 functionality is provided by:

- <IP_NAME>_bcm37
- <IP_NAME>_bcm37_atv

Synchronizer 12 provides a coordinated reset sequence to the source and destination domain logic when a "clear" is initiated by either domain.

Figure 1-29 shows a simple block diagram of Synchronizer 12.

Figure 1-29 Synchronizer 12 Block Diagram



1.14.1 Synchronizer 12 for Automotive IP

Synchronizer 12 functionality for automotive IP is provided by:

- <IP_NAME>_bcm37_atv

In automotive IP, the fundamental concept of meeting ASIL-D requirements is protecting against transient faults of the flip-flops. This is done by the use of triple module redundancy (TMR) replacement for every flip-flip and can include TMR self-test functionality. Note that the TMR logic has no impact on general synchronizer behavior.

For general information about BCMs that support automotive IP, see "TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP" on page 44.

1.15 Synchronizer 13: Simple Reset Synchronizer

Synchronizer 13 functionality is provided by:

- <IP_NAME>_bcm40

Synchronizer 13 offers a solution for synchronizing a reset signal that crosses asynchronous clock boundaries. It allows the versatility of selecting the number of synchronization stages (2, 3, or 4) and includes a multiplexed output to support scan testing (allowing the reset output to be controlled by the scan test scheme while shifting occurs so that registers connected to the output are allowed to shift without being forced to reset when a '0' is shifted through the last stage of the synchronizer).

The block diagrams of Synchronizer 13 in Figure 1-30 and Figure 1-31 are based on the value of the *TST_RST_EN* parameter.

Figure 1-30 Synchronizer 13 Block Diagram *TST_RST_EN* = 0

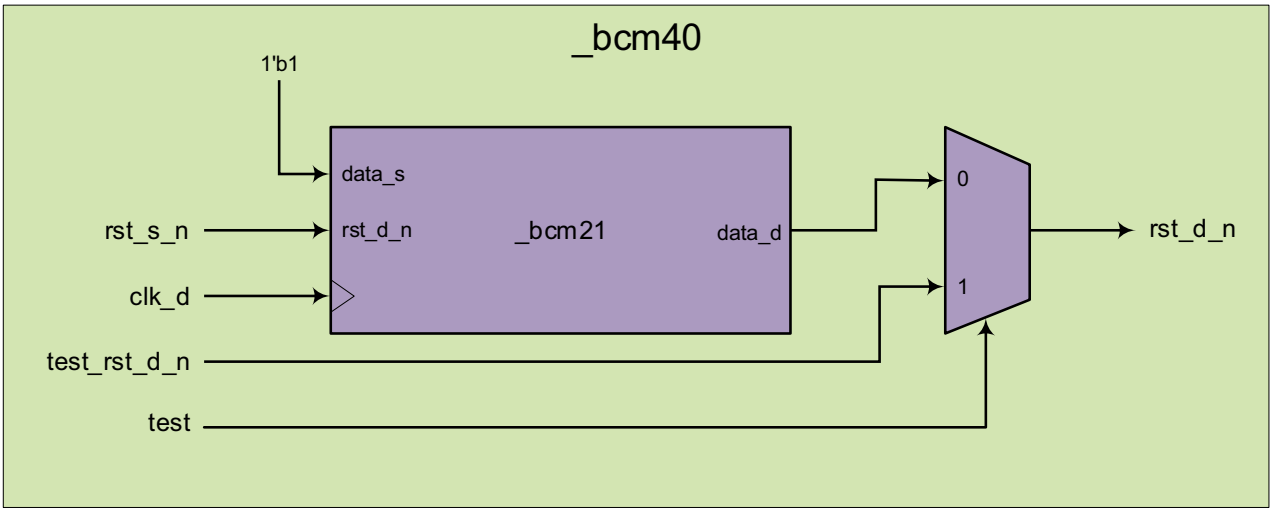
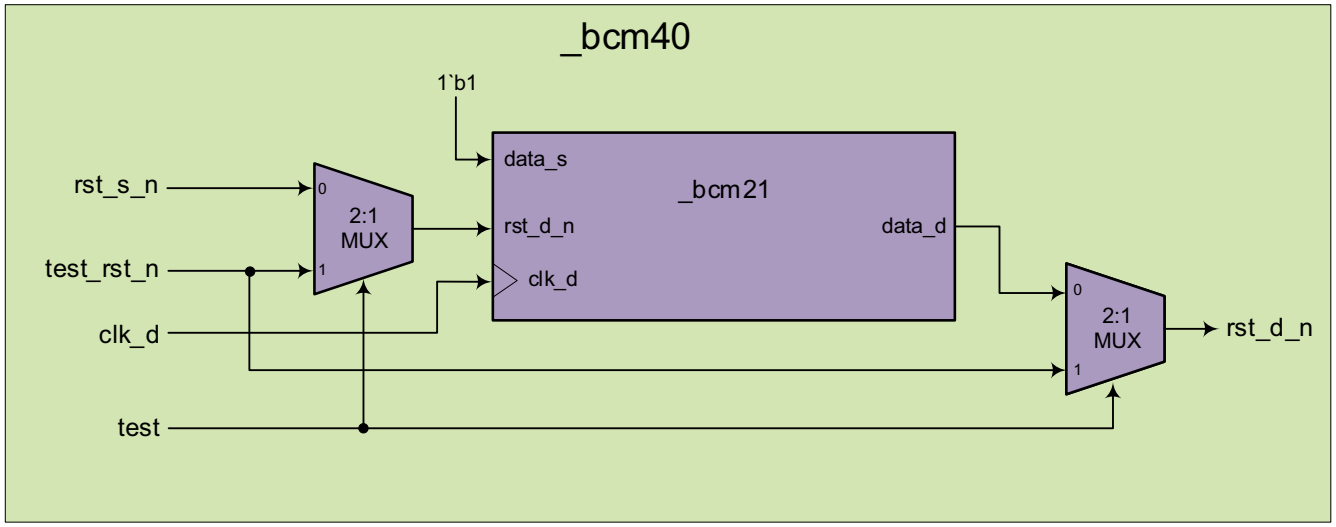


Figure 1-31 Synchronizer 13 Block Diagram *TST_RST_EN* =1



1.16 Synchronizer 14: Reset Synchronizer with Clock Gating

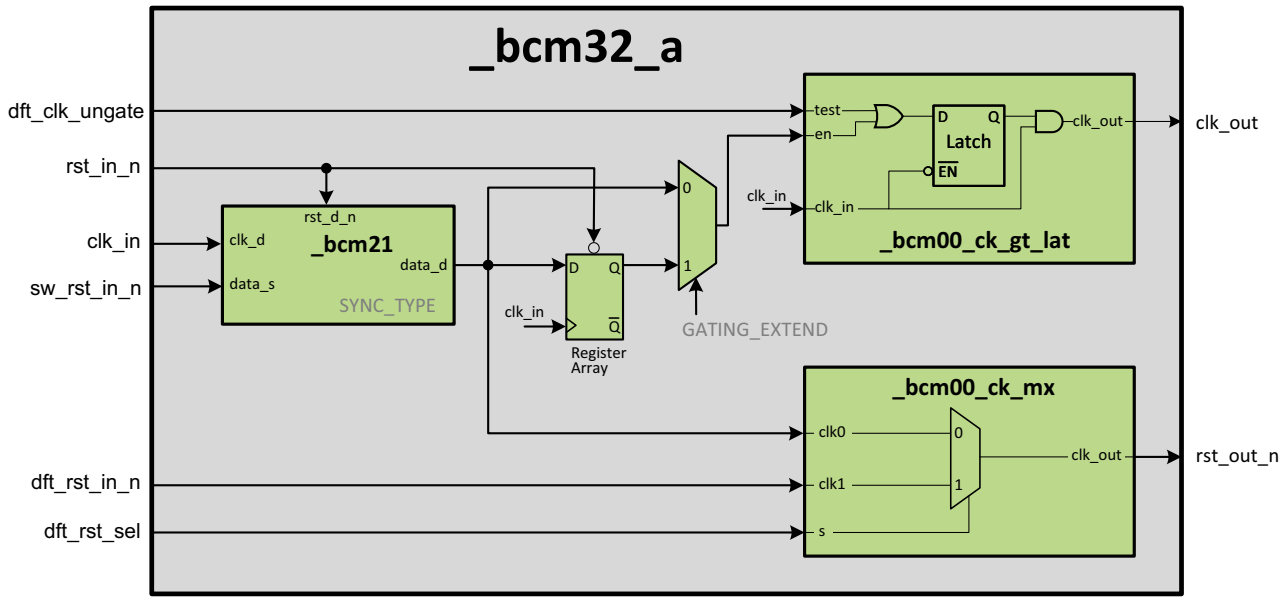
Synchronizer 14 functionality is provided by:

- <IP_NAME>_bcm32_a
- <IP_NAME>_bcm32_c

Synchronizer 14 provides a synchronized reset with a gated clock output. Depending on how it is configured, it can support multiple synchronization stages (2, 3, or 4), hold latches for DFT scan testing, and a reset cycle that is extended by a delay chain of flip-flops. In addition, the <IP_NAME>_bcm32_c allows clock gating to be forced.

Figure 1-32 shows a block diagram for Synchronizer 14 as <IP_NAME>_bcm32_a.

Figure 1-32 Synchronizer 14 Block Diagram for <IP_NAME>_bcm32_a

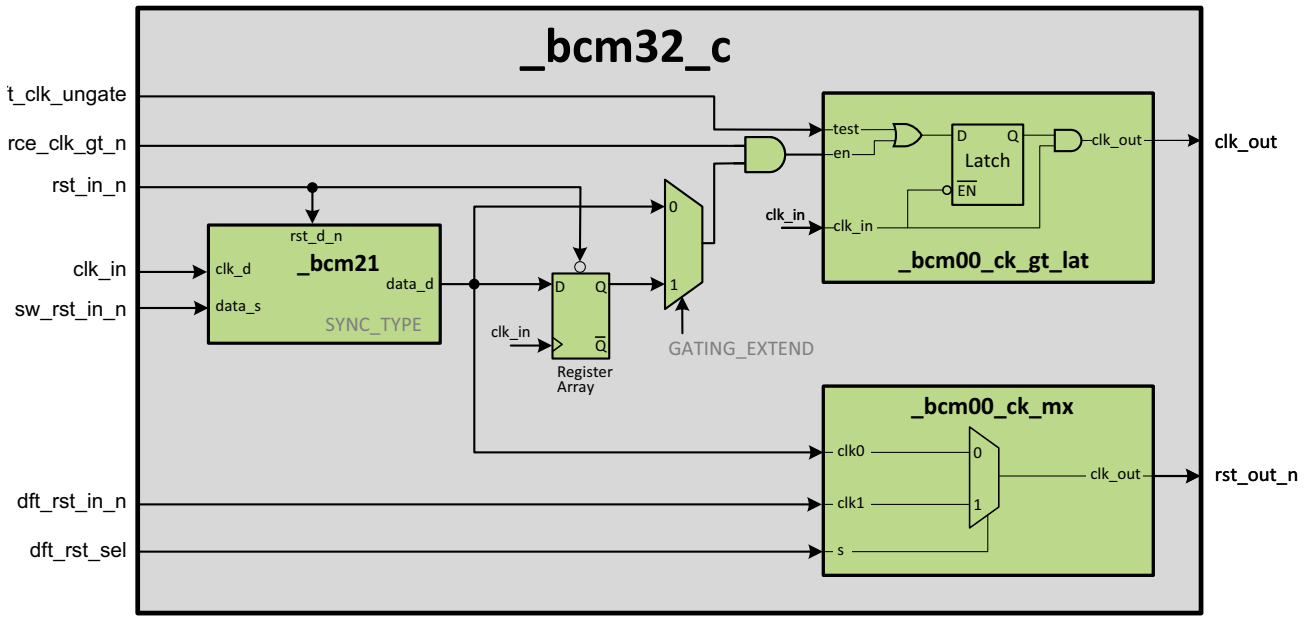


Note the following in both Figure 1-32 and Figure 1-33 on page 41:

- <IP_NAME>_bcm21 implements reset synchronization
- <IP_NAME>_bcm00_ck_gt_lat implements a latch for clock gating for the purpose of scan testing
- <IP_NAME>_bcm00_ck_mx implements a multiplexer for reset selection

Figure 1-33 shows a block diagram for Synchronizer 14 as <IP_NAME>_bcm32_c.

Figure 1-33 Synchronizer 14 Block Diagram for <IP_NAME>_bcm32_c



1.17 Synchronizer 15: Compound Dual Clock FIFO

Synchronizer 15 functionality is provided by:

- <IP_NAME>_bcm79
- <IP_NAME>_bcm79_atv

Synchronizer 15 (<IP_NAME>_bcm79) provides a compound synchronizer using two FIFOs in series. One FIFO (submodule <IP_NAME>_bcm78 with two external RAMs) operates in a single clock domain and the other (submodule <IP_NAME>_bcm66) operates across the clock domains. Submodule <IP_NAME>_bcm07 keeps track of push and pop operations independent of the actual compound FIFOs.

When a data stream is connected, Synchronizer 15 is held in reset (by holding `rst_push_n` and `rst_pop_n` active-low) while the configuration sequence is performed. During this sequence, submodule <IP_NAME>_bcm78 is configured to be in the domain with the faster clock, based on the value of the `push_dom_faster` signal, before releasing the reset inputs for operation of the newly connected stream.

Figure 1-34 shows a block diagram for Synchronizer 15 when the faster clock is in the push domain (`push_dom_faster` = 1).

Figure 1-34 Synchronizer 15 Block Diagram for <IP_NAME>_bcm79 with push_dom_faster = 1

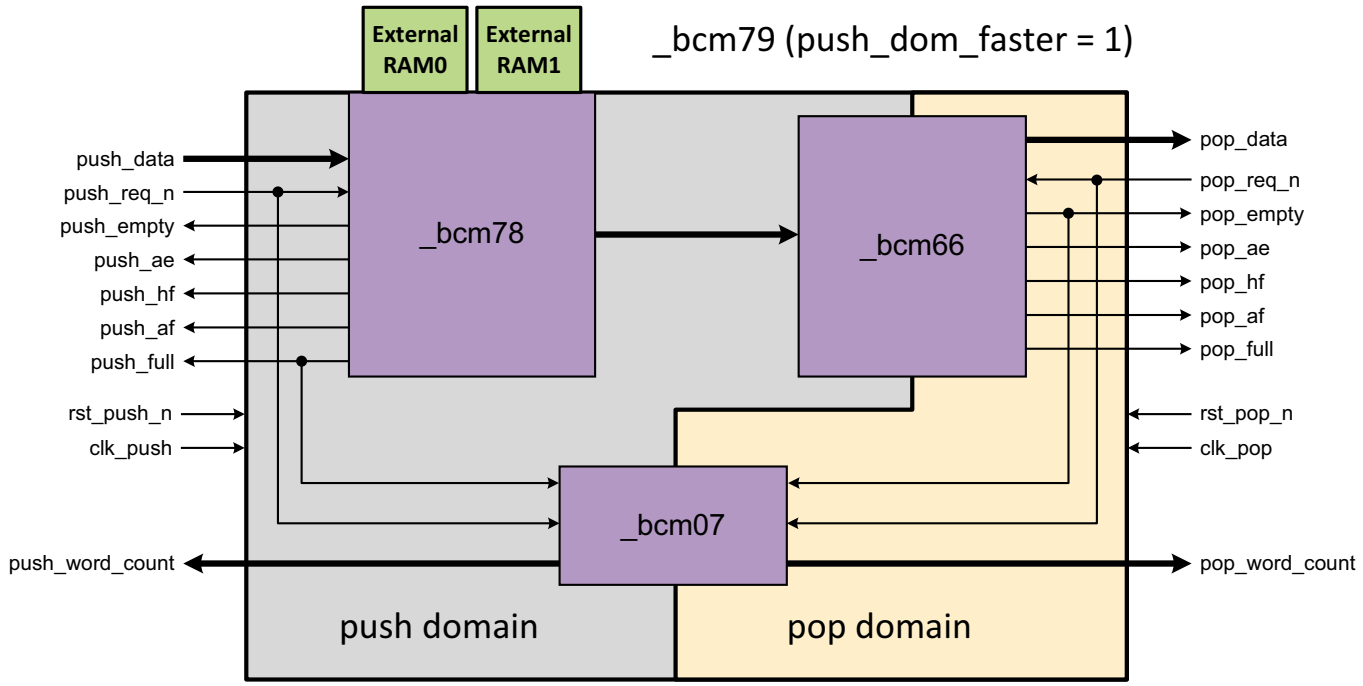
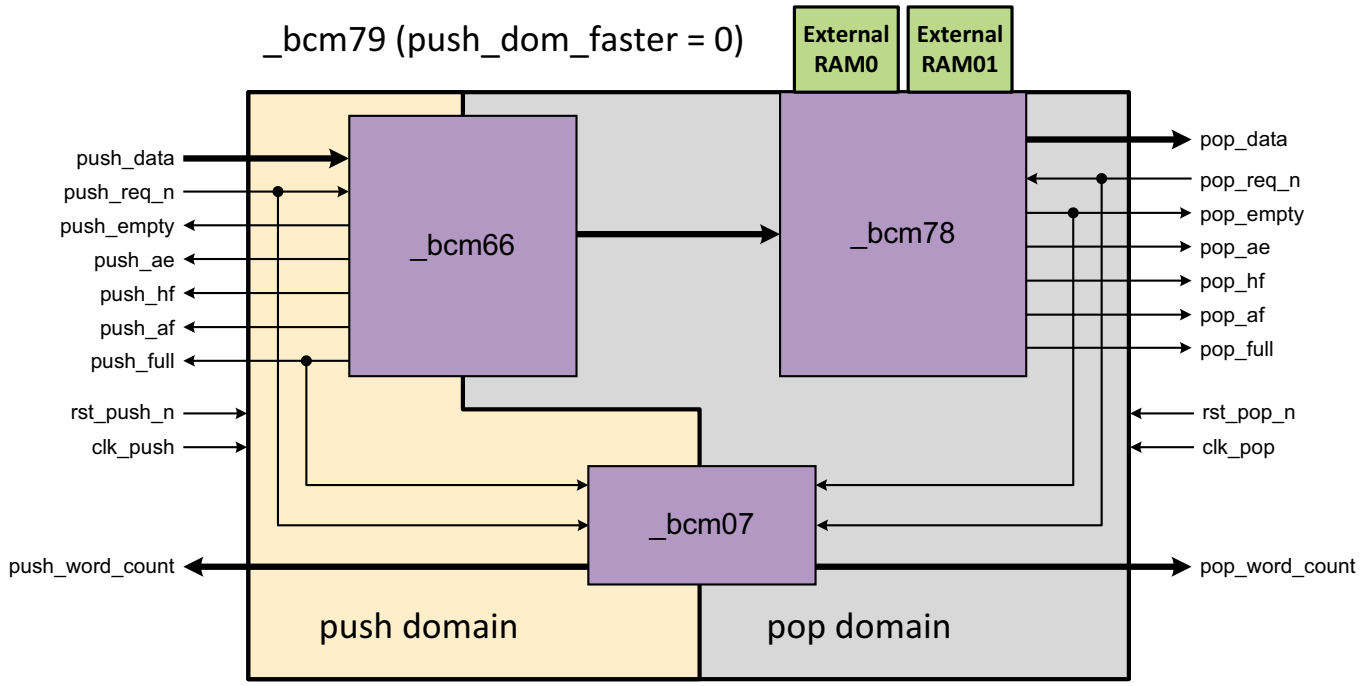


Figure 1-35 shows a block diagram for Synchronizer 15 when the clock in the push domain is slower than the clock in the pop domain (`push_dom_faster` = 0).

Figure 1-35 Synchronizer 15 Block Diagram for <IP_NAME>_bcm79 with push_dom_faster = 0



1.18 TMR Error Reporting and Self-Test Logic In BCM Synchronizers for Automotive IP

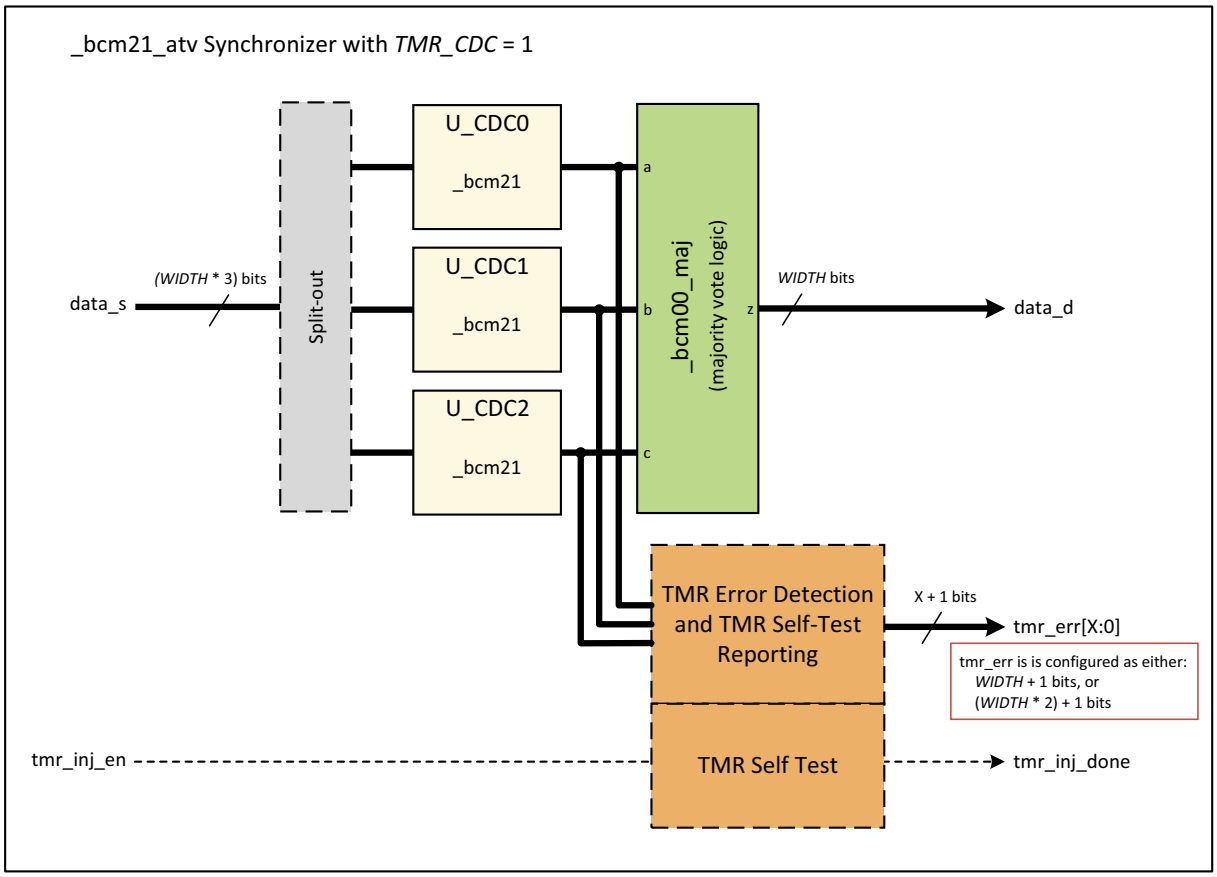
Numerous BCM synchronizers support Automotive IP by implementing triple module redundancy (TMR) logic to protect against transient faults of the flip-flops. Each of these BCMs can include self-test logic to prove the TMR logic is fault-free. The BCMs that support Automotive IP follow a naming convention as follows:

<IP_NAME>_<BCM_NAME>_atv

The synchronizers are replaced by their *_atv equivalent modules, which fundamentally use <IP_NAME>_bcm21_atv for the clock-domain-crossing (CDC) signals. Also, registers that drive signals in the same clock domain for control logic are replaced by <IP_NAME>_bcm95* for the triple redundancy.

Figure 1-36 shows a sample block diagram of <IP_NAME>_bcm21_atv, which provides TMR logic and error reporting for CDC signals when TMR is enabled in the IP. Figure 1-36 also shows a TMR self-test block, which detects and reports errors in the TMR logic.

Figure 1-36 Sample Block Diagram of TMR Error Reporting and Self-Test Logic in <IP_NAME>_bcm21_atv for CDC Signals



Note: TMR Error Detection and TMR Self Test blocks are not configured for all IP

Figure 1-36 shows a sample block diagram of <IP_NAME>_bcm95*, which provides TMR logic and error reporting for registers that drive signals in the same clock domain for control logic. Figure 1-37 also shows a TMR self-test block, which detects and reports errors in the TMR logic.

Figure 1-37 Sample Block Diagram of TMR Error Reporting and Self-Test Logic in <IP_NAME>_bcm95* for Signals In the Same Clock Domain

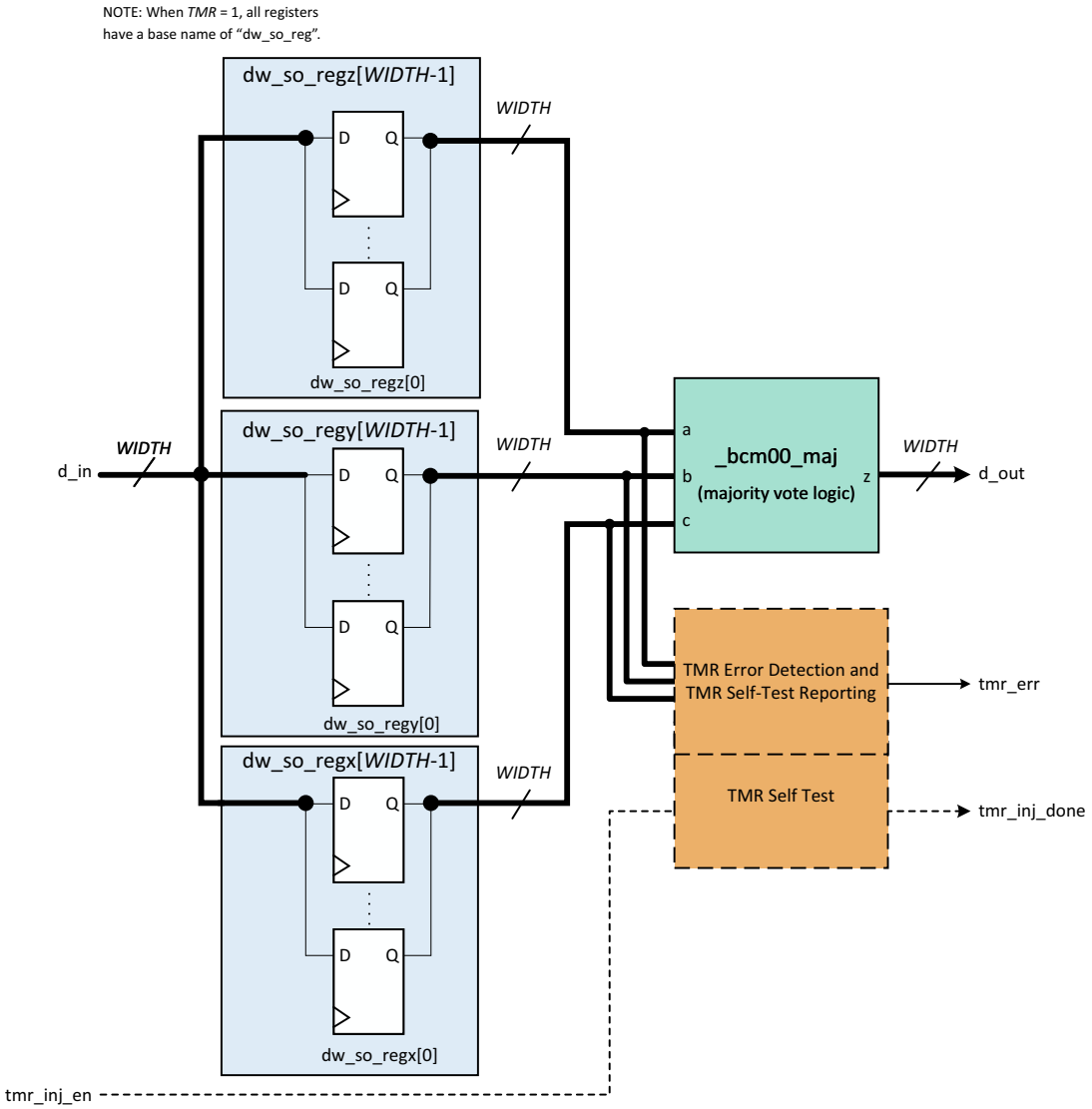


Figure 1-38 shows a sample block diagram of TMR error reporting and self-test logic in IP that needs TMR protection for both CDC signals (with <IP_NAME>_bcm21_atv) and control signals (with <IP_NAME>_bcm95*).

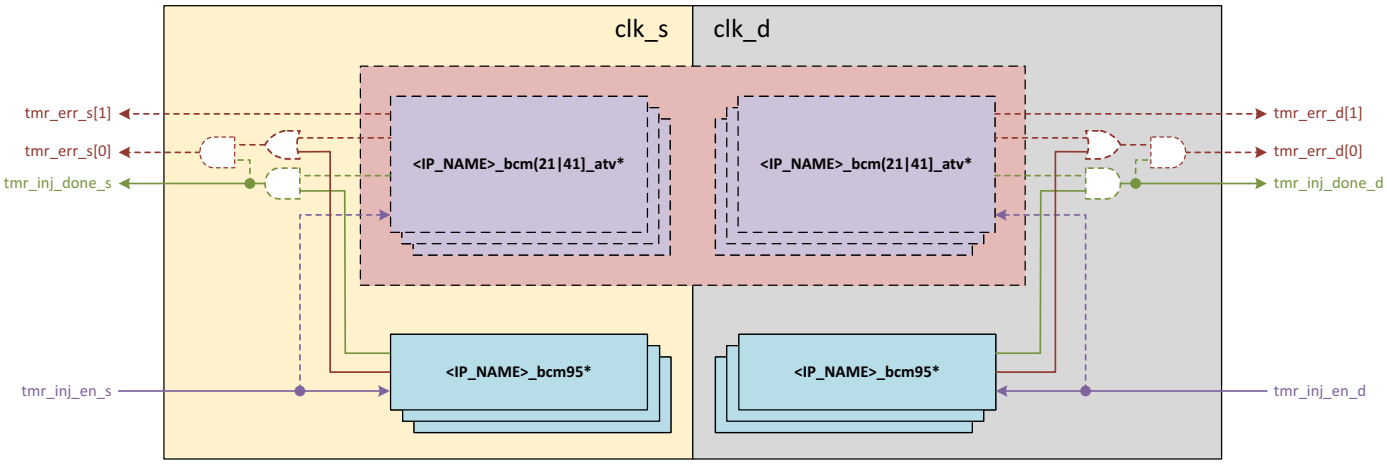



Figure 1-38 Sample Block Diagram of TMR Error Reporting and Self-Test Logic